Highlights

Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni,Marco Pagliocca,Enrico Vicario,Roberto Verdecchia

- One specification, 67 automated checks: 7 stacks run the same experiment
- 210 runs, 12600 blocks; training energy spans 7.8×–18.2× across stacks
- Among 3 stacks running identical kernels the spread is 1.1×–3.8×
- The estimator's duration field, not the stacks, produced "faster is not greener"
- 67 % of blocks are timed over a longer window than their energy

Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni[a,*], Marco Pagliocca[a,*], Enrico Vicario[a] and Roberto Verdecchia[a]

[a] University of Florence, Florence, Italy

ARTICLE INFO

Keywords:
Machine Learning
Deep Learning
Green AI
Energy Efficiency
Sustainability
Green IT

ABSTRACT

Context. Deep Learning's environmental footprint raises a practical question: does the choice of language–framework ecosystem carry an energy price? Answering it requires that every ecosystem run the same experiment and be measured the same way — neither of which is verifiable from reported energy figures alone.

Objectives. We quantify how ecosystem choice affects the energy efficiency of DL training and inference, and, inseparably, what an apparatus must guarantee before such a comparison means anything.

Methods. Two convolutional architectures (ResNet-18, VGG-16) across 7 ecosystems — Python with PyTorch, TensorFlow and JAX; C++ with LibTorch; Java with Deeplearning4j; R with `torch`; Rust with `tch` — on three datasets. A written specification defines what "the same experiment" means across stacks and 67 automated checks enforce it against the source. Energy is read from hardware counters (NVML and RAPL) with CodeCarbon over the same phase boundaries as a second reading. Each configuration runs 5 independent interleaved times, and every stack records per-epoch accuracy.

Results. Over 210 runs and 12600 doubly instrumented blocks, training energy differs across ecosystems by 7.8× to 18.2× depending on the block, with a median between-run coefficient of variation of 0.44 %. Among the 3 stacks executing a byte-identical TorchScript module the spread is 1.1×–3.8×, so much of the effect is binding and host-side work rather than kernels. On Fashion-MNIST the stacks converge to within 1.3 percentage points; on the harder datasets they do not, and energy must be read alongside accuracy. Replication exposed a failure a single-run design cannot see: VGG-16 reaches exactly chance accuracy in 12 of 105 runs, consuming 10.7 % of the campaign's energy for nothing. On the apparatus, the estimator's energy agrees with the counters to 0.5 % because on this platform it reads the same registers — but 67 % of blocks are reported with a duration seconds longer than the interval their energy was accumulated over, so power derived from those two fields is understated by up to 11.2× on the shortest blocks and correct on the longest.

Conclusions. Ecosystem selection materially affects the energy profile of DL applications, but the size and even the direction of that effect depend on decisions in the measurement apparatus that are invisible in the resulting data. We report the comparison together with the specification, the conformance checker and the raw dual-instrument records needed to audit it.

1. Introduction

The rapid advancement of Artificial Intelligence (AI), particularly in the domain of Deep Learning (DL), has delivered groundbreaking progress across domains such as healthcare, natural language processing, environmental monitoring, and autonomous systems. As models grow in size and complexity, their performance continues to improve, often reaching or surpassing human-level capabilities in specific tasks. However, these achievements have come at a growing environmental cost. The training and deployment of state-of-the-art neural networks demand vast computational resources and energy, contributing to significant carbon footprints [1, 2, 3, 4, 5, 6, 7]. Recent studies show that energy usage and emissions vary widely depending on model architecture, dataset size, hardware infrastructure, and even geographic location of execution [8, 9]. In response, standardized benchmarks, such as MLPerf Tiny, have begun to incorporate energy measurements to assess efficiency under realistic deployment conditions [10]. In a world of limited environmental resources, the ever-growing energy required to power AI has become a pressing concern, making sustainability a central dimension of AI research [11, 12].

Rather than framing this trend solely as a cause of concern, the field is now embracing a proactive paradigm, referred to as *Green AI* [12, 13]. This research line promotes the development of AI systems that are not only accurate and effective but also energy-conscious and efficient. Recent surveys have highlighted advances across lightweight architectures, efficient training and inference strategies, model compression, and hardware-aware optimizations [4, 14, 15, 16]. Evidence increasingly points in the direction of energy improvements not necessarily implying a degradation of accuracy [17].

Beyond algorithmic efficiency, research in software sustainability has shown that the choice of programming language itself can have a substantial impact on energy usage [18, 19, 20, 21, 22, 23, 24], and on the run time and memory that partly determine it [25]. Nevertheless, while both Green AI and the environmental sustainability of programming languages have been independently investigated, their intersection remains largely unexplored. A first step

*Corresponding authors: Leonardo Pampaloni and Marco Pagliocca.
leonardo.pampaloni1@edu.unifi.it (L. Pampaloni); marco.pagliocca@edu.unifi.it (M. Pagliocca); enrico.vicario@unifi.it (E. Vicario); roberto.verdecchia@unifi.it (R. Verdecchia)
ORCID(s): 0000-0001-9206-6637 (R. Verdecchia)

was made by Georgiou *et al.* [26], who analyzed the energy footprint of Deep Learning frameworks within Python, and by Marini *et al.* [27], who studied the impact of programming languages in AI workloads.

For a practitioner, however, a programming language is inseparable from its *ecosystem*. Choosing a language for DL typically implies adopting a specific software stack, including language bindings, frameworks/libraries, runtimes/execution engines, and low-level driver/toolchain dependencies. As a result, in real-world settings it is often more meaningful to evaluate *language–framework ecosystems* as they are adopted in practice, rather than attempting to isolate a "pure" language effect under synthetic, fully standardized conditions.

Prior work has approached this from three directions without closing it. Energy has been compared across DL frameworks *within* one language ecosystem, including at the GPU and runtime-infrastructure level [26, 28]; language-level energy has been measured for CPU-bound general-purpose workloads [18, 21]; and the *runtime* cost of cross-language DL bindings has been characterised for time, memory and accuracy, but not for energy [29]. What has not been measured is energy across multiple language-based DL stacks for GPU training and inference, where frameworks, execution engines, toolchains and dataset complexity all vary together [30].

This paper addresses this gap by presenting an empirical study on *the impact of language–framework ecosystems on the energy efficiency of DL workloads*. To achieve our goal, we consider two widely adopted convolutional neural networks (ResNet-18 and VGG-16) across 7 language–framework ecosystems, spanning five host languages (Python, C++, Java, R and Rust) and the frameworks each of them offers in practice (PyTorch, TensorFlow and JAX; LibTorch; Deeplearning4j; and the `torch` and `tch` bindings), on three heterogeneous datasets (Fashion-MNIST, CIFAR-100 and Tiny ImageNet).

A cross-ecosystem energy comparison, however, rests on two premises that are rarely stated and never visible in the resulting numbers: that every stack is running the same experiment, and that every stack is measured the same way. We found both to be far harder to satisfy than the literature suggests, and we came to that conclusion the expensive way. An earlier campaign of ours, of exactly this shape, was executed carefully and produced an entirely coherent story; auditing it against its own source code afterwards showed that the stacks had drifted apart in learning rate, input shape, evaluation batching and data-loader parallelism, that the measurement tool had been configured five different ways across them, and that two stacks were capable of running the whole workload on the CPU while reporting nothing but a log line. None of this was visible in the energy figures. All of it moved them.

We therefore treat the apparatus as part of the contribution rather than as a preliminary. Three commitments follow. *(i)* A written experiment specification states what "the same experiment" means across stacks — model, optimisation, data pipeline, backend, measurement and replication — and 67 automated checks enforce it against the source code, so that a divergence fails a check instead of quietly changing a number. *(ii)* Energy is read from hardware counters, NVML's accumulated-energy register for the accelerator and the RAPL package-energy counters for the CPU, with CodeCarbon running over the same window as a second, independent instrument; every block therefore carries two readings that can be held against each other. *(iii)* Each configuration is executed as 5 independent, interleaved runs with distinct seeds, so that the run rather than the epoch is the unit of analysis, and every ecosystem records per-epoch test accuracy, so that energy can be normalised by the quality actually reached rather than assumed equal.

Our results are correspondingly more modest and more defensible than a bare ranking. Between-run repeatability is high — the median coefficient of variation of training energy across 210 runs is 0.44 % — so the intervals we report, Student-t on five independent runs, describe between-run uncertainty rather than epoch autocorrelation. Under a common measurement boundary, training energy differs across ecosystems by 7.8× to 18.2× depending on architecture and dataset, with JAX consistently cheapest, while final test accuracy converges to within 1.3 percentage points on Fashion-MNIST across all 7 stacks. On the two harder datasets the stacks do not converge to a common accuracy, and there energy must be read alongside quality rather than instead of it. The ecosystem effect is real and large; on Fashion-MNIST it is energetic rather than qualitative.

Replication earned its cost in a way we did not anticipate. VGG-16 fails to train at all in 12 of 105 runs — converging to exactly chance accuracy and staying there for the full epoch budget — in 4 of the 7 ecosystems independently, and never once for ResNet-18. Those runs consume 10.7 % of the campaign's energy and produce nothing, and they are invisible in energy data alone. They also account for almost all of the apparent accuracy disagreement between stacks: excluding them takes the cross-ecosystem spread on VGG-16 on CIFAR-100 from 20.2 to 1.3 percentage points. A design with one run per configuration reports whichever of the two outcomes its single seed produced.

The two readings agree on *energy* to 0.5 % over 12600 blocks — on this platform the estimator reads the same two registers we do, so its energy is not an estimate at all, and that relocates the difficulty with software-based estimation rather than dissolving it. What does not agree is time. The duration CodeCarbon reports beside each energy figure is not the interval the energy was accumulated over: 67 % of blocks carry seconds of tracker lifetime that no energy was drawn in, in 3 discrete modes that block length predicts but does not determine. Power derived by dividing the one field by the other is therefore understated by up to 11.2× on the shortest blocks and correct on the longest — a bias that lands squarely on the fastest ecosystems and on the inference phase, which is to say on precisely the comparisons such a study exists to make.

The contributions of this work are as follows:

- A systematic empirical comparison of DL energy consumption across 7 language–framework ecosystems for GPU-based workloads, replicated 5 times per configuration and reported with between-run uncertainty and quality normalisation;
- An experiment specification for cross-ecosystem energy studies, together with an executable conformance checker that enforces it, and a single measurement contract implemented by all 7 stacks;
- A dual-instrument characterisation of CodeCarbon against hardware energy counters over 12600 measurement blocks, including a quantified failure mode of its reported duration that invalidates energy–time analyses built on it;
- A comprehensive replication package, including all data, scripts, and settings required to fully reproduce our results.[1]

Improving the energy efficiency of AI systems is not just a technical challenge but also an ethical imperative as the field matures and scales [31]. In this regard, providing concrete empirical evidence on the energy consumption of different language–framework ecosystems represents a crucial first step towards the development of AI that is not only powerful, but also truly sustainable.

2. Background

The notion of *Green AI* [12, 13, 17] promotes the development of AI systems that are not only accurate but also energy-efficient and sustainable. Several indicators can be used to quantify sustainability, such as carbon footprint, financial cost, or energy usage [7, 5]. In this work we focus on **energy consumption**, measured in Joules, as it represents a direct and reproducible measure of computational effort. Unlike carbon footprint, which is heavily influenced by the energy mix and geographic location of execution, energy consumption can be compared more consistently across heterogeneous DL stacks under a uniform measurement protocol.

We treat execution time as a secondary metric rather than a proxy for energy. The prevailing position in the software-energy literature is that lower runtime does not imply lower energy [30, 32, 18], and we test it directly in Section 6.4 rather than assuming it. Our result differs from that literature, and the difference is instructive: on counter-bracketed durations, energy and time rank our configurations almost identically. Those studies largely concern CPU-bound general-purpose benchmarks, where a language implementation can trade instruction count against memory traffic and shift energy per second substantially. In a GPU-bound DL workload the accelerator dominates and holds a narrow band of power, so time and energy are close to proportional. We report this as a scope condition on the established finding, not a refutation of it — and we show in Section 6.5 that a study of exactly our shape can obtain either answer depending on which field of the instrument's output it reads as the duration.

Validating the instrument is itself an active line of work. Comparative studies run software estimators alongside hardware wattmeters and report close tracking under controlled conditions and larger deviations on heterogeneous infrastructure [33, 34]; independent validation against external ground truth across hundreds of AI experiments reports estimator errors up to 40% and systematic underestimation of 20–30% in some settings [35]. That work asks whether an estimator's number is close to the truth.

We ask a different and complementary question: whether a study can *tell* when it is not. The distinction matters because the two lead to different remedies. If the problem is accuracy, the remedy is a better estimator or a correction factor. If the problem is that an estimator silently substitutes a model for a measurement, pads a duration it reports as measured, or degrades below its own sampling interval, then no correction factor helps: what is needed is a second instrument over the same window, so that agreement becomes an observable property of each block. Our findings in Section 6.5 suggest the second framing is the more useful one for cross-ecosystem work, since on this platform the estimator's *energy* turned out to be accurate and its *duration* turned out not to be a duration.

Deep Learning workloads are typically divided into two distinct phases: training and inference. *Training* requires iterative updates of model parameters through backpropagation and is generally the most computationally expensive stage. *Inference*, by contrast, involves only forward passes on a trained model and is usually less resource demanding. Both phases, however, display different energy profiles—*i.e.* optimizations in one phase do not necessarily carry over to the other [14, 15]. This motivates the need to analyze the two phases separately when evaluating their energy efficiency.

Language–framework ecosystems may influence the energy efficiency of DL workloads through a combination of factors, including framework execution engines, runtime overheads, data input pipelines, numerical precision policies, and the underlying driver/toolchain configuration. While compiled and interpreted languages differ in their execution models, in GPU-based DL workloads much of the computation is typically delegated to optimized backend kernels; therefore, differences observed in practice should be interpreted at the *ecosystem level* rather than attributed to the programming language alone. Moreover, some ecosystems (e.g. MATLAB with its Deep Learning Toolbox, or Java with Deeplearning4j) introduce additional runtime layers compared to lower-level bindings (e.g. LibTorch in C++ or `torch` in R), which can affect both performance and energy behavior. Understanding these ecosystem-level differences is crucial, since language choice is often treated as a secondary engineering decision, yet in practice it constrains the set of available frameworks, runtimes, and supported

[1] Replication package: `https://github.com/Pampaj7/DeepGreen`. The Zenodo record `https://zenodo.org/records/17734884` archives the earlier campaign and is superseded by this one.

toolchains, ultimately affecting the energy profile of DL model training and inference.

Energy measurement for workloads of this kind can be read at three places, and the choice of place matters more than the choice of tool. A *wall meter* sees the whole machine including power supply losses, fans and drives. *Hardware energy counters* — NVML's `nvmlDeviceGetTotalEnergyConsumption` on NVIDIA accelerators and Intel RAPL on the CPU package — accumulate energy in the silicon itself and are read by software but not estimated by it. *Software estimators* such as `CodeCarbon` [36] sit on top of whatever the platform exposes and substitute a model wherever it exposes nothing.

Software estimators have become standard in sustainability-oriented AI research [31, 8], and are relied upon as the sole instrument across model compression analysis [37], optimizer efficiency benchmarks [38], large-scale evaluations of Large Language Models [39] and cybersecurity systems [40] — studies whose conclusions inherit whatever the estimator gets wrong, without a second reading that could reveal it. Comparative studies have also run `CodeCarbon` alongside hardware wattmeters in the same setup and report that software estimates track sensor readings closely under controlled conditions while deviating more on heterogeneous infrastructure [33, 34]. Independent validation against external ground truth across hundreds of AI experiments nonetheless reports estimator errors of up to 40%, and systematic underestimation of 20–30% in some settings [35].

We take a different position from the one this literature usually implies. The question is not whether a software estimator is accurate enough to be trusted, but whether a study can tell when it is not. An estimator that falls back on a model when a counter is unavailable, and announces the substitution only in a log line, produces a number of the same shape and plausibility either way. We therefore instrument every measurement block *twice*: hardware counters as the primary instrument, and `CodeCarbon` started and stopped at the same phase boundaries as a second reading. Agreement between the two is then an observable property of each block rather than an assumption, and the places where they cannot agree — CodeCarbon's RAM term is derived from installed memory capacity, and no counter on this platform can confirm or refute it — are identified rather than absorbed.

This also fixes the boundary. NVML plus RAPL is a *board-and-package* measurement: the accelerator card including its memory, plus the CPU package, excluding the power supply, fans, drives and host DRAM that a wall meter would see. It must not be presented as a whole-system figure. We had no wall meter available and say so, reporting a board-and-package quantity consistently rather than a mixture of measured and modelled terms that corresponds to no physical boundary at all. Section 6.5 reports what the two instruments say about each other across the campaign.

3. Related Work

Table 1 places this study against the work closest to it on the two axes that matter here: how many language ecosystems are compared, and how energy is measured.

Two of these deserve more than a row. Alizadeh and Castor [28] compare runtime infrastructures for the same models on the same GPU, and reach the ecosystem-level question from inside Python: their treatment is the runtime beneath one language, ours is the language–framework stack above it. Li *et al.* [29] compare TensorFlow and PyTorch bindings across five languages, which is our independent variable exactly, and measure run time, memory and accuracy rather than energy — the gap this study fills, and the reason we treat the measurement boundary as a contribution rather than a preliminary.

Jay *et al.* [41] compare software power meters, CodeCarbon among them, against physical meters on CPU and GPU. Their finding that a measurement window must comfortably exceed a meter's sampling interval is the precedent for a constraint we ran into independently and report in Section 6.5; what we add is a characterisation of the *duration* an estimator reports beside its energy, which to our knowledge has not been examined.

4. Research Methodology

This work adopts an empirical methodology grounded in the principles of software engineering empirical experimentation [42, 43, 44, 45]. Our objective is not to propose a new algorithm or framework, but to systematically analyze how choosing among language–framework ecosystems (including their supported runtime and toolchain) can impact the energy efficiency of DL workloads. To this end, we designed an *in vitro* controlled experiment that treats ecosystems as independent variables, DL workloads as experimental objects, and energy consumption as the primary dependent variable [46, 47, 48].

Research goal and questions. Following the Goal-Question-Metric (GQM) paradigm [46], our goal is formulated as:

> *Analyze* DL training and inference,
> *for the purpose of* quantifying and comparing their energy consumption,
> *with respect to* the choice of DL ecosystems (i.e. language bindings, frameworks/libraries, supported runtimes/execution engines, and associated toolchains),
> *from the viewpoint of* software engineering researchers and practitioners,
> *in the context of* computer vision image classification workloads.

Research questions. This study investigates how language–framework ecosystem selection influences the energy efficiency of Deep Learning (DL) workloads. We focus on three key questions:

Table 1
The closest prior work, on the two axes this study varies. "Stacks" counts distinct language–framework combinations compared under one protocol.

Study	Stacks	Hardware	Energy instrument	Replication
Pereira *et al.* [18]	27 languages, no DL	CPU	RAPL	10 runs
Gordillo *et al.* [21]	8 languages, no DL	CPU	hardware meter	repeated
Georgiou *et al.* [26]	2 frameworks, Python only	CPU + GPU	RAPL + NVML	30 runs
Alizadeh & Castor [28]	4 runtimes, Python only	CPU + GPU	software estimator	repeated
Li *et al.* [29]	5 bindings, 2 frameworks	CPU + GPU	none (time, memory, accuracy)	repeated
Jay *et al.* [41]	software meters, not stacks	CPU + GPU	meters against a wattmeter	repeated
Marini *et al.* [27]	5 languages	CPU + GPU	software estimator	single run
This study	7 stacks, 5 languages	GPU	NVML + RAPL counters, estimator alongside	5 runs

- **RQ1:** *How does language–framework ecosystem choice affect the energy efficiency of DL* training*?*
- **RQ2:** *How does language–framework ecosystem choice affect the energy efficiency of DL* inference*?*
- **RQ3:** *How do energy efficiency and execution time relate across different DL ecosystems?*
- **RQ4:** *To what extent are the answers to RQ1–RQ3 determined by the measurement apparatus rather than by the ecosystems?*

RQ4 is not a robustness check appended to the other three. A cross-ecosystem energy comparison is an inference from an instrument to a claim about software, and the inference is only as good as the guarantee that the instrument reads the same quantity, over the same window, for every stack. We therefore treat the apparatus as an object of measurement in its own right: every block is recorded by two independent instruments, and RQ4 asks what they say about each other and about the answers to RQ1–RQ3.

Objects of study. We focus on convolutional neural networks (CNNs), which remain fundamental in modern computer vision tasks [4, 14, 15]. Specifically, we consider two canonical architectures: ResNet-18 [49] and VGG-16 [50]. These models are selected not only for their distinct computational characteristics (residual connections vs. sequential depth), but primarily because they are natively provided *off-the-shelf* by the vast majority of DL frameworks. By relying on these standard, pre-implemented architectures, we ensure that our measurements capture the frameworks' internal optimizations and native performance, rather than introducing researcher-specific biases that would result from implementing the models from scratch.

To account for varying workload complexity, we employ three widely utilized datasets. *Fashion-MNIST* provides a lightweight baseline with 28×28 grayscale images across 10 classes [51], and is often used as a modern drop-in replacement for MNIST[2]. It includes 60,000 training and 10,000 test images. *CIFAR-100* increases complexity with 32×32 color images across 100 categories, remaining a canonical benchmark for image classification [52], and comprises 50,000 training and 10,000 test images. *Tiny ImageNet* represents a demanding workload with 64×64 color images across 200 classes and 100,000 training and 10,000 test images, serving as a proxy for ImageNet-scale tasks [53].

All datasets were resized to a uniform resolution of 32×32 pixels. This preserves the relative differences in complexity (grayscale vs. color, number of classes, dataset size) while removing input resolution as a confounding factor [27, 26]. In addition, standardizing input size avoids the need to modify the final classification layer of the networks for each dataset, thus keeping model architectures fixed across experiments. This design choice not only improves comparability of results but also reflects realistic developer practice, where pre-trained or canonical models are often reused with minimal architectural modifications [17, 13]. Finally, a common resolution improves stability across frameworks, some of which (e.g. `tch` for Rust, Deeplearning4j for Java) offer more reliable performance with standardized image sizes [18, 21]. Nevertheless, fixing a 32×32 input resolution may interact with framework-specific memory and kernel selection strategies; therefore, conclusions should not be extrapolated to higher-resolution workloads without additional experiments.

Independent and dependent variables. Our primary independent variable is the *DL ecosystem*, i.e. the language–framework stack as adopted in practice (including its officially supported runtime and CUDA/cuDNN toolchain). In our design, each ecosystem is treated as a single categorical treatment; we do not attempt to decompose effects into separate contributions of language, framework, runtime, or toolchain. Specifically, we evaluate 7 language-based ecosystems: Python (`PyTorch`, `TensorFlow`, `JAX`), C++ (`LibTorch`), Java (`Deeplearning4j`), R (`torch`), and Rust (`tch`). A second independent variable is the *DL phase* (training and inference).

MATLAB with the Deep Learning Toolbox was part of an earlier campaign and is out of scope here. The reason is specific rather than dismissive: the toolbox does not expose the training loop at the granularity the specification of Section 4.1 requires, so its data pipeline, its evaluation batching and its measurement boundaries cannot be brought into conformance with the other 7. Including a stack that provably runs a different experiment would reintroduce exactly the confound this design exists to remove.

[2] https://github.com/zalandoresearch/fashion-mnist

Our dependent variables are the *energy consumption* of each measurement block, in Joules at a board-and-package boundary (accelerator card plus CPU package), and the *test accuracy* reached at the end of each epoch. Recording the second alongside the first is what allows energy to be normalised by achieved quality in Section 6.2; a fixed-budget comparison that does not record accuracy cannot distinguish an efficient stack from one that has simply learned less [5, 31, 6].

Study design. We adopt a *black-box design*: models are executed using off-the-shelf implementations under identical hyperparameters (e.g. learning rate, batch size) [20, 19, 32]. This choice enforces the realism of the study by reflecting how practitioners use frameworks *in vivo*, rather than relying on ad-hoc or reimplemented versions.

By keeping hyperparameters constant, we implement a controlled *fixed-budget* comparison across ecosystems (i.e. identical training recipe and number of epochs). This design ensures a fair baseline comparison in which we minimize the risk of introducing researcher-specific implementation biases, thereby improving comparability; however, it does not guarantee convergence equivalence across frameworks. This represents an explicit trade-off: allowing per-ecosystem hyperparameter tuning could yield better absolute efficiency for each stack, but would confound whether differences stem from the ecosystem itself or from the tuning process.

Moreover, we treat framework-level execution engines and optimizations (e.g. graph compilation, runtime schedulers, backend kernels) as *part of the ecosystem* rather than normalizing them away. As a consequence, observed differences should be interpreted as ecosystem-level effects, not as isolated language-level behavior.

Numerical precision policy. We retained the default numerical precision and mixed-precision policies of each ecosystem (e.g. FP32 vs mixed precision), treating them as part of the ecosystem-level behavior observed in practice. Consequently, differences in precision paths may contribute to measured energy and execution time; we therefore avoid attributing observed effects to the programming language alone.

4.1. What "the same experiment" means, and how it is enforced

A fixed-budget comparison presumes that the budget is the same. In practice each ecosystem is written by a different person against a different API, and the places where they can silently diverge are numerous, individually reasonable and invisible in the energy output: a default learning rate that differs between two framework APIs, a data loader whose worker count follows the machine rather than the protocol, an evaluation loop written one image at a time because the API made that natural, an input tensor left at its native resolution in one stack and resized in another.

We therefore wrote the protocol down before running anything, as a specification with six parts, and made conformance machine-checkable.

- **S1 — Model.** The 3 ecosystems that share the pinned LibTorch build (Python, C++, Rust) train the *same* TorchScript module, exported once per (architecture, dataset) pair, rather than three independent reimplementations of ResNet-18 and VGG-16. R links a LibTorch of its own and its binding cannot switch a script module between training and evaluation mode, so it builds an equivalent architecture instead; it belongs to the LibTorch lineage but not to the control group. The remaining stacks use the canonical architecture their framework ships, with parameter counts checked against the exported module.
- **S2 — Optimisation.** One optimiser, one learning rate, one batch size, one epoch count, one loss, one seeding policy, applied identically and asserted at startup.
- **S3 — Pipeline.** One input specification ($32 \times 32 \times 3$, raw $[0, 1]$ inputs, no per-stack normalisation), a bounded and equal number of loader workers, and batched evaluation at the training batch size in every stack.
- **S4 — Backend.** The accelerator is asserted at startup and CPU fallback is refused rather than warned about. The LibTorch version is pinned across the four stacks that share it.
- **S5 — Measurement.** One measurement bridge, one sampling interval, one tracking mode, one set of counters, one unit, and per-epoch accuracy persisted alongside energy.
- **S6 — Replication.** Each configuration is executed 5 times as independent processes with distinct seeds, interleaved rather than run back to back.

The specification is enforced by 67 automated checks that read the source of every ecosystem and the configuration of every tracker, and fail the build when a stack drifts out of conformance. This matters more than it may appear. A specification that lives only in a paper is a description of intent; the same specification expressed as an executable check is a property of the artefact, and it fails at the moment of the divergence rather than at peer review. 5 of the defects catalogued in Section 8 were introduced by us during development, and every one was caught by a mechanism rather than by inspection. Two of them are now themselves checks in this list, so the same divergence cannot recur.

Residual, documented divergences. Two could not be removed and are reported rather than hidden. Deeplearning4j 1.0.0-M2.1 is the last release of its API and links against the CUDA 11.6 runtime, so the Java stack cannot be brought onto the CUDA 12 toolchain the other six share. And the R `torch` binding cannot switch a script module between training and evaluation mode, so R alone cannot consume the shared TorchScript module of S1 and builds an equivalent architecture instead. Both are properties of the ecosystems, and arguably part of what one measures when one measures an ecosystem, but they are not the binding overhead the study sets out to isolate.

4.2. Measurement procedure

Energy is read from hardware counters at the boundaries of each measurement block: NVML's `nvmlDeviceGetTotalEnergyConsumption`, an accumulating millijoule register on the accelerator, and the RAPL package-energy counters exposed at `/sys/class/powercap` for the CPU, with wraparound at `max_energy_range_uj` handled explicitly. The reported quantity is their sum, identical for every ecosystem.

The two halves of that sum are not at the same boundary, and the sum is not "the chip". NVML's register is a *board* quantity: it includes the GPU die, its GDDR6X and the card's voltage regulators. RAPL's package domain is the CPU package alone. Host DRAM is in neither — this platform exposes no `dram` RAPL domain, so its energy, of order ten watts, is not measurable here and is absent from every figure we report. We name the quantity board-plus-package rather than chip-level, and we note the asymmetry with our own criticism of the estimator's modelled RAM term: that term is unverifiable, and ours is missing altogether.

`CodeCarbon` [36] runs over the same window as an independent second instrument, in machine tracking mode at a one-second sampling interval, configured identically for all 7 stacks through a single shared bridge. The four non-Python ecosystems drive that bridge over a line protocol with synchronous acknowledgement, so that a phase boundary in the measured process and the corresponding counter read cannot drift apart — an earlier asynchronous version of the same bridge under-reported one stack's training energy by 24% and recorded its evaluation energy as zero.

A measurement block is one phase of one epoch: 12600 blocks in total, each carrying two energy readings, a duration, and the test accuracy reached. Because tracking is machine-wide, no other work runs on the host during a campaign; blocks measured while a background download was in progress during development were discarded and the protocol amended rather than corrected after the fact.

Replication and the unit of analysis. Each of the 42 configurations is executed 5 times as an independent process with a distinct seed, giving 210 runs. The repetitions are *interleaved* rather than run consecutively, so that drift in machine state — thermal, background, driver clock behaviour — is spread across conditions instead of aliasing onto whichever ecosystem happened to run last. All uncertainty we report is between-run: the 30 epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, and treating them as repeated measurements would understate uncertainty while inflating apparent significance.

Quality normalisation. Because every stack records test accuracy per epoch, we report energy in three forms: energy per epoch, energy per unit of accuracy achieved, and the energy required to first reach a target accuracy fixed per dataset. The last is the closest to a practitioner's question, and is the one a fixed-budget design cannot answer on its own.

Reproducibility and rigor. We fixed pseudo-random seeds for data shuffling and model initialization where supported by the ecosystem. Nevertheless, full determinism cannot be guaranteed across all stacks due to non-deterministic GPU kernels and differences in backend execution engines; this is one reason the design replicates rather than relying on a single execution per configuration.

A complete replication package is provided (see replication package link in Section 1), including: (i) source code for all implementations, (ii) scripts to reproduce runs, (iii) environment specifications (e.g. virtual environments, Maven, CMake), and (iv) collected raw data [17, 13]. Configurations are standardized across languages where feasible, with deviations documented when framework-specific requirements arise [18, 21]. This package enables replication, validation, and extension under different hardware or software conditions [42, 44].

5. Experimental Execution

This section describes the practical execution of the study, complementing the research design presented in Section 4. Here we document the experimental setup, implementation details, measurement procedure, and reproducibility package, following a step-by-step process to ensure transparency [42, 45].

Experimental setup. All experiments reported here were executed on a single dedicated workstation: an NVIDIA GeForce RTX 3090 (Ampere, 24 GB GDDR6X, 350 W board power limit, driver 595.84), an AMD Ryzen 9 5900X (12 cores / 24 threads), 30 GB of RAM and NVMe storage, running Ubuntu 26.04 LTS (kernel 7.0.0). The machine was otherwise idle for the duration of the campaign, which whole-machine energy tracking requires.

Two properties of this platform bear directly on the measurements. First, both energy counters are available: the RTX 3090 exposes NVML's accumulated-energy register, and the Ryzen's package-energy counters are readable through the Linux `powercap` interface — which, for historical reasons, presents AMD domains under `intel-rapl` names. We read the package domain and handle counter wraparound at `max_energy_range_uj` explicitly. Second, this is a desktop rather than a server: it has one accelerator, one CPU socket and an order of magnitude less RAM than a datacentre node, which is precisely why the modelled RAM term discussed in Section 6.5 is a smaller share of a software estimator's total here than it would be on a large-memory host.

We report this as a replication platform rather than as the platform. An earlier campaign of the same design ran on a dual-socket Intel Xeon Gold 5418Y server with an NVIDIA L40S; absolute energy figures are not comparable between the two machines, and none of the comparisons in this paper is made across them. What carries over is the design, the specification and the instrument characterisation, all of which are properties of the method rather than of the host.

Model implementations. Every ecosystem uses the canonical architecture its framework provides rather than a reimplementation, and, where the backend permits, the *same* artefact. Under specification S1 the four stacks on the pinned LibTorch build — Python, C++ and Rust — do not build three independent ResNet-18s and VGG-16s but load one TorchScript module per (architecture, dataset) pair, exported once and version-pinned in a manifest that the conformance checker compares against the LibTorch each stack links. R links its own bundled LibTorch and builds an equivalent architecture, for the reason given under S1. This removes what would otherwise be the largest uncontrolled variable in the study: in an earlier campaign these four stacks differed in parameter count, and comparing them was therefore comparing four models, not four bindings.

Table 2 lists the versions. The Python stacks are installed in three separate virtual environments rather than one, because their CUDA and cuDNN wheels conflict: a shared environment resolved to a cuDNN that TensorFlow could not dlopen, and TensorFlow then ran unaccelerated while reporting only a warning — a failure mode the accelerator assertion of S4 now turns into an error.

Toolchain divergence, and what is left of it. The four LibTorch stacks are pinned to one LibTorch build (2.7.0, CUDA 12.8), so within that control group the backend is genuinely held constant. Two ecosystems cannot join it. Deeplearning4j 1.0.0-M2.1 links against CUDA 11.6 and cuDNN 8.3 and has had no subsequent release; the stack now carries its own redistributables through `org.bytedeco:cuda-platform-redist`, which makes it reproducible without making it contemporary. R's `torch` 0.17.0 bundles its own LibTorch and resolves CUDA at load time. We report these as documented divergences rather than normalising them away, and Section 6.2 reports the LibTorch control group separately so that a reader can see how much of the total spread survives when the backend is held fixed.

During a preliminary pilot phase, the Julia programming language (using `Flux` and `Lux`) was also considered. However, persistent technical challenges encountered during implementation related to gradient updates during training, which remained unresolved also after directly contacting the developers of the libraries, prevented its reliable inclusion in the final study.

Procedure. As summarized in Table 3, each experimental configuration (model × dataset × ecosystem) is executed with consistent hyperparameters tuned to balance comparability and realism [4, 14]. Training runs are performed for 30 epochs with a batch size of 128, using the Adam optimizer with a fixed learning rate of 1×10^{-4}. Loss is computed via cross-entropy, consistent across all ecosystems. Input preprocessing includes resizing all images to 32×32 pixels, with grayscale datasets (e.g. Fashion-MNIST) expanded to three channels for compatibility with CNN backbones. These settings reflect canonical practices in the community while ensuring comparability across implementations [10, 18].

Measurement. Each phase of each epoch is bracketed by two counter reads — NVML's accumulated-energy register and the CPU package RAPL counter — and is simultaneously tracked by `CodeCarbon` v3.3.0 [36]. The two are started and stopped by the same code path, so their windows cannot drift apart. They are not identical: the counter reads are nested immediately inside the tracker's, which costs a fixed offset we quantify in Section 6.5 rather than assume away.

The four non-Python ecosystems drive the instrumentation through a single shared bridge: a Python process that accepts `START`, `STOP`, `METRIC` and `EXIT` over a line protocol, reads the counters, and acknowledges each command before the measured process proceeds. The acknowledgement is not incidental. An earlier, asynchronous version of the same bridge allowed a phase boundary in the measured process to run ahead of the corresponding counter read, which underreported one stack's training energy by 24% and recorded its evaluation energy as exactly zero — a value that looks like a bug only if someone thinks to look at it.

Running the bridge out of process also isolates the measured stack from the instrument: a crash in the tracker cannot take down a six-hour training run, and the tracker's own CPU cost is attributed to the machine total rather than silently to the workload. All runs execute in isolation on an otherwise idle host, as whole-machine tracking requires, and each block is logged with dataset, model, ecosystem, repetition, seed, phase, epoch, both energy readings, the duration and the test accuracy reached.

Reproducibility. A comprehensive replication package accompanies this study (see the link in Section 1). Beyond source code, automation scripts, per-ecosystem environment specifications and raw logs, it contains three artefacts specific to this design:

(i) the **experiment specification** of Section 4.1, written before the campaign and versioned with it;

(ii) the **conformance checker**, 67 executable checks that read the source of every ecosystem and the configuration of every tracker, and fail when a stack drifts out of specification;

(iii) the **dual-instrument records**: every block's counter reading and estimator reading side by side, which is what makes the analysis of Section 6.5 reproducible rather than anecdotal.

Every quantity this manuscript reports from the replicated campaign is generated from those records by the analysis pipeline and injected into the source at build time. The exceptions are stated where they occur: figures recovered from the audit of the earlier campaign, and one-off diagnostics reproduced from the record that established them, which are not recomputable from the campaign because they describe a campaign that no longer exists.

6. Results

We report the campaign in the order the research questions were posed, after a subsection on repeatability that

Table 2
Ecosystems, frameworks, versions and backends. The 3 stacks marked † share one pinned LibTorch build and one exported TorchScript module, and form the internal control group of the study. R, marked ‡, links a LibTorch of its own and builds an equivalent architecture: same lineage, different backend and different artefact.

Ecosystem	Framework / Library	Version	Backend
Python	PyTorch / TorchVision†	v2.7.0 / v0.22.0	LibTorch 2.7.0, CUDA 12.8
	TensorFlow / Keras	v2.19.1 / v3.15.1	cuDNN 9.3.0, CUDA 12 wheels
	JAX / Flax / Optax	v0.10.2 / v0.12.8 / v0.2.8	cuDNN 9.24.0, CUDA 12 wheels
C++	LibTorch†	v2.7.0	LibTorch 2.7.0, CUDA 12.8
Java	Deeplearning4j	v1.0.0-M2.1	ND4J CUDA 11.6, cuDNN 8.3
R	`torch`‡	v0.17.0	bundled LibTorch, CUDA 12
Rust	`tch`†	v0.20.0	LibTorch 2.7.0, CUDA 12.8
Instrumentation			
	CodeCarbon	v3.3.0	identical config, all stacks
	NVML / RAPL counters	driver 595.84	read by the shared harness

Table 3
Hyperparameter configuration, identical across all ecosystems and enforced by the conformance checks of specification S2 rather than by convention.

Parameter	Value
Number of epochs	30
Batch size	128
Optimizer	Adam
Learning rate	1×10^{-4}
Loss function	Cross-entropy
Input resolution	32×32 (all datasets)
Input channels	3 (grayscale datasets expanded)
Accelerator	asserted at startup; CPU fallback refused
Data loader workers	2, bounded and equal in every stack
Input normalisation	none; raw $[0, 1]$ inputs everywhere
Evaluation	batched at the training batch size
Independent runs	5 per configuration, interleaved

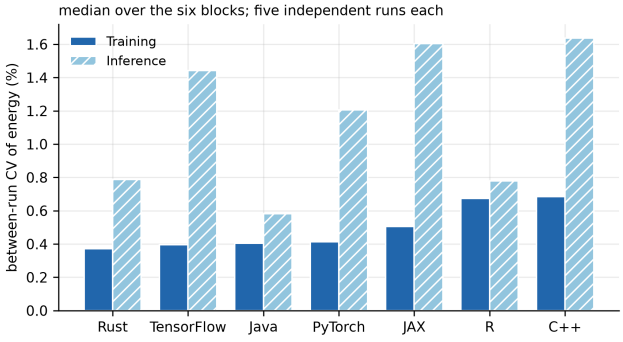

Figure 1: Between-run coefficient of variation of energy, median over the six (model, dataset) blocks, from 5 independent runs per configuration.

bounds everything after it. One departure is worth flagging: RQ4, on the apparatus, comes last but conditions everything before it, and a reader who wants to know how much to trust Sections 6.2–6.4 should read Section 6.5 first.

All figures below are between-run. Each configuration contributes 5 independent runs, and every interval, test and effect size is computed across those runs rather than across the epochs within one.

6.1. Repeatability of the measurements

Before comparing ecosystems it is worth asking how precisely the apparatus measures any single one of them, because the answer bounds every comparison that follows and because the submitted design could not produce it at all.

Across all 42 configurations, the median between-run coefficient of variation of training energy is 0.44 % and the worst is 1.48 %. Inference, whose blocks are one to two orders of magnitude shorter, is noisier but still tight: a median of 1.15 % and a worst case of 9.56 %. Figure 1 breaks this down by ecosystem.

Two things follow. First, differences of a few percent between ecosystems are resolvable, and the differences we report below are far larger than that. Second, the run-to-run variability is small enough that a single run per configuration would have produced a plausible-looking number — which is precisely why a single run per configuration is not evidence that the number is right. Low variance is a property of the measurement, not a guarantee about the thing measured.

6.2. RQ1: Impact of ecosystems on training energy efficiency

Figure 2 and Table 4 report training energy per epoch for every ecosystem and block, at a common board-and-package boundary.

The spread between the cheapest and the most expensive ecosystem ranges from 7.8× to 18.2× depending on the block (Table 5), with JAX cheapest in every one. This is a substantially larger effect than the 4.6× an epoch-averaged, single-run, mixed-boundary analysis of a campaign of this shape would report, and it is larger for a straightforward reason: once every stack is measured at the same boundary, the constant host term that used to be added to all of them — compressing their ratios toward one — is gone.

Figure 2: Training energy per epoch, accelerator plus CPU package, with 95% Student-t intervals over 5 independent runs. Note the logarithmic axis: the intervals are narrow enough to be hard to see at this scale.

Table 4
Mean training energy per epoch (J), accelerator plus CPU package.

Ecosystem	ResNet-18			VGG-16		
	CIFAR-100	F-MNIST	Tiny-IN	CIFAR-100	F-MNIST	Tiny-IN
JAX	581	640	1 773	2 013	2 409	4 080
C++	905	1 077	2 122	4 869	5 842	9 834
TensorFlow	927	1 096	2 310	3 119	3 719	6 279
Rust	1 202	1 333	4 554	5 373	6 278	13 821
PyTorch	3 392	4 080	6 874	5 229	6 277	10 555
Java	8 311	9 965	16 821	15 904	19 114	31 896
R	8 661	11 670	18 054	8 225	11 187	17 823

Table 5
Cheapest and most expensive ecosystem per block, training energy per epoch.

Model	Dataset	Lowest		Highest	
		J		J	Ratio
ResNet-18	CIFAR-100	JAX	581	R	8 661 (14.9×)
ResNet-18	Fashion-MNIST	JAX	640	R	11 670 (18.2×)
ResNet-18	Tiny ImageNet	JAX	1 773	R	18 054 (10.2×)
VGG-16	CIFAR-100	JAX	2 013	Java	15 904 (7.9×)
VGG-16	Fashion-MNIST	JAX	2 409	Java	19 114 (7.9×)
VGG-16	Tiny ImageNet	JAX	4 080	Java	31 896 (7.8×)

The shared-backend control group. 3 of the 7 ecosystems — Python, C++ and Rust — train the byte-identical TorchScript module on the same pinned LibTorch build under specification S1. Whatever separates *those three* cannot be the kernels, because the kernels are the same code; it is binding overhead, data movement and host-side dispatch. Within that control the spread is 1.1× to 3.8× (Table 6), against 7.8× to 18.2× across all 7. The answer splits by architecture, and we state it rather than leaving the table to speak. On ResNet-18 the control group spans 3.2×–3.8× and accounts for 51 % of the log spread: binding and host-side work are a first-order term there. On VGG-16 the control spans only 1.1×–1.4× and accounts for as little as 4 %: the three stacks running identical kernels are nearly indistinguishable, and the remaining spread is the frameworks that do not share those kernels. ResNet-18 is the smaller network, so a larger share of its epoch is host-side dispatch — which

Table 6
Spread within the shared-module control group against the full spread, per block. The control holds the LibTorch build, the exported module and the data pipeline fixed. The LibTorch-family column adds R, which shares the lineage but neither the build nor the module.

Model	Dataset	All 7	Shared module (3)	LibTorch family (4)	Share of log spread
ResNet-18	CIFAR-100	14.9×	3.8×	9.6×	49%
ResNet-18	Fashion-MNIST	18.2×	3.8×	10.8×	46%
ResNet-18	Tiny ImageNet	10.2×	3.2×	8.5×	51%
VGG-16	CIFAR-100	7.9×	1.1×	1.7×	5%
VGG-16	Fashion-MNIST	7.9×	1.1×	1.9×	4%
VGG-16	Tiny ImageNet	7.8×	1.4×	1.8×	17%

Table 7
Final test accuracy (%) after the fixed epoch budget, mean over the 10 runs of each cell (both architectures, 5 repetitions each). Where runs collapsed to chance, the mean over the runs that trained follows in parentheses.

Ecosystem	Fashion-MNIST	CIFAR-100	Tiny ImageNet
C++	91.30	30.98	11.60 (14.37)
Java	91.03	20.27 (28.52)	10.23 (12.66)
JAX	91.36	30.56	15.19
PyTorch	91.19	31.23	12.99 (14.37)
TensorFlow	90.98	22.79 (28.24)	11.31 (14.02)
R	91.64	33.34	16.57
Rust	91.12	30.81	14.74

is the mechanism the split implies and which we cannot confirm without profiling we did not do.

R is reported beside the control rather than inside it. It links its own bundled LibTorch and, because the R binding cannot switch a script module between training and evaluation mode, builds an equivalent architecture instead of loading the shared one — so it shares neither the build nor the module. Including it widens the "shared-backend" spread to 1.7×–10.8×, which would have been reported as binding overhead and is not: the difference between the two columns of Table 6 is the cost of one member that shares less than the label claims. We made that mistake in an earlier version of this analysis and record it here rather than quietly correcting it.

Energy at equal quality. A fixed-budget comparison rewards a stack for learning less. Because every ecosystem now records test accuracy per epoch, we can check whether it is doing so. Table 7 reports final test accuracy by ecosystem and dataset. On Fashion-MNIST all 7 stacks land within 1.3 percentage points of one another on either architecture, which is the strongest available evidence that they are running the same experiment — and a check the submitted design could not perform, because no ecosystem wrote its accuracy to disk. We quote the per-architecture figure deliberately: pooling ResNet-18 and VGG-16, which reach different accuracies in opposite per-ecosystem directions, would report 0.7 points and would be measuring the pooling rather than the stacks. On the harder datasets the stacks separate more — to at most 7.5 points per block once the runs that failed to train are excluded (Section 6.2.1) — and there the energy comparison must be read alongside the accuracy reached rather than instead of it.

Table 8
Median training energy (kJ) to first reach a target accuracy, with the number of runs that reached it where that is not all 10. Targets: 85 %, 30 %, 20 %.

Ecosystem	Fashion-MNIST *85%*	CIFAR-100 *30%*	Tiny ImageNet *20%*
C++	3	43 (9/10)	never
Java	24	231 (2/10)	never
JAX	3	19 (5/10)	41 (5/10)
PyTorch	5	34	never
TensorFlow	6	39 (3/10)	88 (1/10)
R	17	51	110 (5/10)
Rust	4	28	never

Figure 3 plots the two together. Ranking ecosystems by accuracy per kilojoule rather than by energy alone puts JAX ahead of Java by 13.6× — 11.3× counting only the runs that trained — a different quantity from the energy ratio, and closer to the one a practitioner choosing a stack actually faces. That pooled figure averages values two orders of magnitude apart, since accuracy per kilojoule on Fashion-MNIST and on Tiny ImageNet are not the same kind of number, so we give it per dataset as well: 13.0× on Fashion-MNIST, 15.9× on CIFAR-100 and 12.3× on Tiny ImageNet, with JAX best on all three.

Energy to a target, and what it costs to report it. Closest of all is the energy a stack spends before it first reaches a stated accuracy, which a fixed-budget design cannot produce. Table 8 reports it against a target fixed per dataset. On Fashion-MNIST every run of every ecosystem reaches 85 %, and the energy to get there spans 7.1× — a like-for-like comparison at equal quality, and the cleanest number in this paper. On the harder datasets the table is mostly informative for its gaps: 4 of the 7 stacks never reach 20 % on Tiny ImageNet within the epoch budget at all, so for those cells there is no energy-to-target to report and the fixed-budget columns are the only comparison available. We show the empty cells rather than dropping them, because "did not reach the target" is the result.

6.2.1. Runs that consume the budget and learn nothing

Replicating each configuration surfaced something a single run cannot show. VGG-16 does not always train. We call a run *collapsed* when its final test accuracy never exceeds 1.5× chance for its dataset; the multiplier is generous on purpose, because every collapsed run in this campaign sits at chance exactly — 1/100 on CIFAR-100, 1/200 on Tiny ImageNet — so no case depends on where the line is drawn. By that criterion 12 of 105 VGG-16 runs collapse and stay collapsed for all 30 epochs, having consumed the full energy budget while learning nothing. ResNet-18 never does this: 0 of 105 runs. Neither architecture does it on Fashion-MNIST.

The per-epoch traces say precisely what these runs are. In every one of them the loss sits at $\ln N$ for the N classes of the dataset — 4.605 on CIFAR-100, 5.298 on Tiny ImageNet — to within 0.0007, for all thirty epochs. That is the loss of

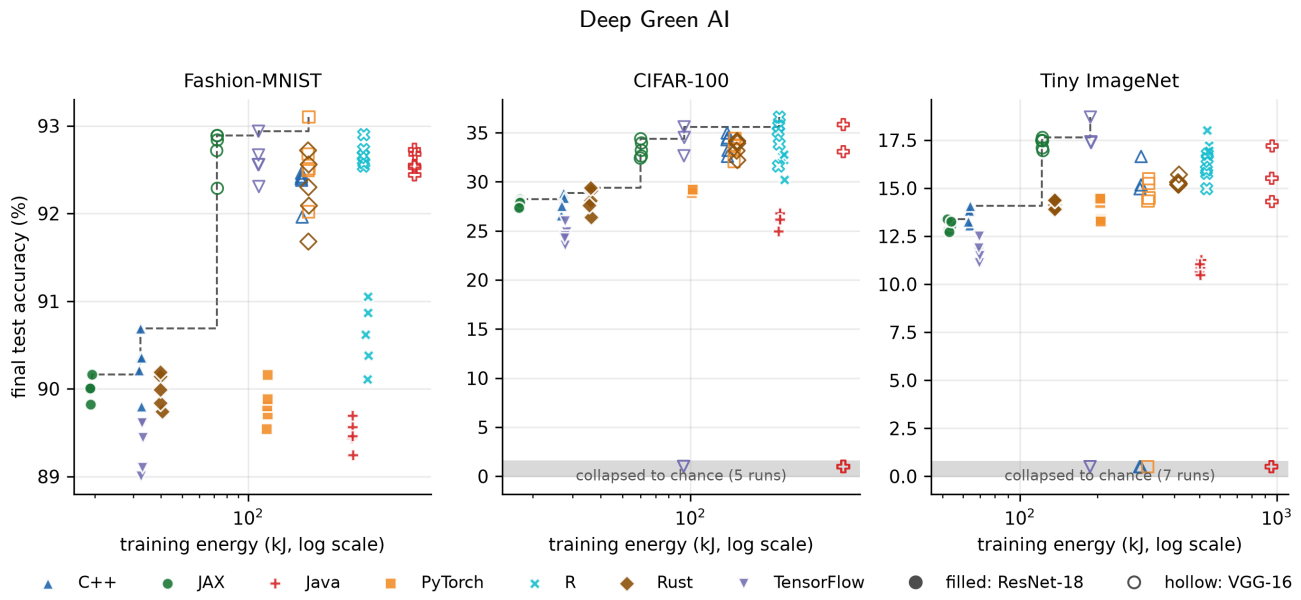

Figure 3: Energy spent against accuracy reached, one point per run. The dashed line is the empirical frontier: no configuration to its left reaches the same accuracy for less energy.

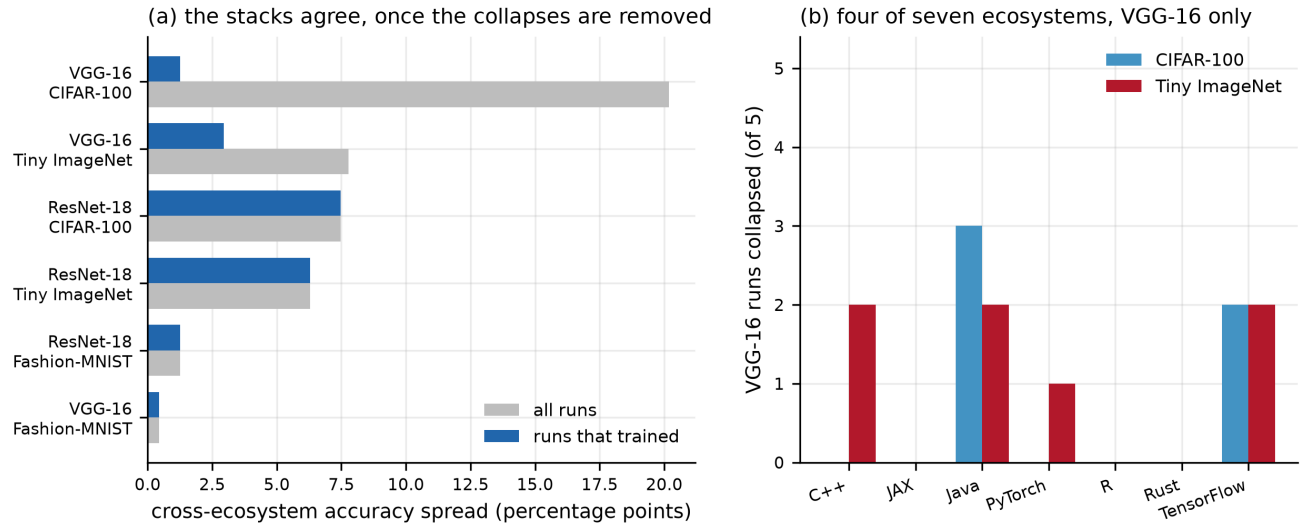

Figure 4: (a) Cross-ecosystem accuracy spread per block, over all runs and over only the runs that trained. (b) VGG-16 runs that collapsed to chance, out of five per cell.

a *uniform* output distribution: the network is not confidently predicting one class, it is producing the same distribution for every input, and no gradient signal is reaching it. VGG-16 as shipped carries no batch normalisation, and a plain sixteen-layer network driven by Adam at 10^{-4} over a hundred or two hundred classes can start there and never leave.

What decides whether a run does so is not the initial weights. The 3 stacks that load the byte-identical exported module start from identical parameters, and they disagree about which repetitions collapse: in 2 cells one of them settles at chance while another trains to full accuracy on the same architecture, dataset and repetition index. What differs between them is the stochastic path — shuffle order under each framework's own generator, and non-deterministic kernels — not the starting point. The collapse appears in 4 ecosystems independently, and 3 never exhibit it. Per-ecosystem rates range from 0 % to 50 %, against an overall 17 % of the runs where collapse occurs at all — VGG-16 on the two many-class datasets, ten runs per ecosystem. That spread is large, and ten runs per ecosystem are not enough to establish it: the two tests we ran disagree, for reasons we set out in Section 9. We therefore report the susceptibility as a property of the recipe, report that the ecosystems plausibly differ in rate, and decline to say which are more robust. Resolving it needs more repetitions than an energy study can usually afford, and it is a question worth someone's time: the answer would be about each framework's generator and shuffle order, not about its kernels.

A failed recipe and a broken pipeline look identical in an energy table. Both produce chance accuracy at full

energy cost, and neither is visible in a Joule figure. The per-epoch traces separate them completely. In a genuine collapse the training loss does not move either — across all 12 of them it changes by at most 0.0 %, because a network that has settled on one class has nothing left to fit. A pipeline defect looks the opposite: the training loss falls normally while test loss *rises* and accuracy stays at chance, which says that training and evaluation are not seeing the same inputs.

We report this because the discriminator found a defect of our own that the collapse count alone would have absorbed. 4 runs met the chance-accuracy criterion — at most 14.7 % — with training loss falling by 86–87 %: one Rust binary carried a private copy of the input transform, applied on the training path only, which resized the tensor its loader had already produced using a function that returns `uint8`. Every value in $[0, 1]$ truncated to zero, so it trained on black images and was evaluated on real ones. Filed as collapses they would have read as "VGG-16 sometimes fails to train".

They were a defect, it is fixed, and the configuration was re-executed: those runs now reach the same accuracy as every other stack, and the discriminator finds 0 pipeline defects in the campaign as it stands. The 12 collapses reported above are the ones that survive the test. Since the evidence for a defect should be as traceable as the evidence for a result, and the discarded runs no longer exist in the campaign, their signature is preserved as a record in the replication package rather than quoted from memory.

We are able to state the discriminator's recall on the one instance we have, and it is not perfect. The defect was a deterministic code path on the training branch, so all 5 repetitions of that configuration carried it; the discriminator flagged 4. The remaining run finished above the collapse threshold and so was never a candidate for the test at all. A discriminator calibrated on a single positive example, with a threshold chosen after seeing it, is a heuristic and not a detector: it separates the two failure modes cleanly once a run is known to have failed, and it will miss a defect that leaves enough accuracy behind. We report it as the diagnostic aid it is.

Two consequences matter here.

It is the strongest evidence that the specification worked. On VGG-16 on CIFAR-100 the cross-ecosystem accuracy spread is 20.2 percentage points over all runs and 1.3 points over the runs that trained (Figure 4a). Almost the entire apparent disagreement between stacks was a handful of runs that failed to train, not a difference in what the stacks compute. No block exceeds 7.5 points once collapses are excluded.

It breaks fixed-budget energy comparison, invisibly. A collapsed run costs full energy and produces nothing: 4.8 MJ, or 10.7 % of the campaign's 45.0 MJ of training energy, was spent on runs that learned nothing. An ecosystem that drew unlucky initialisations looks equally expensive and much less accurate; one that drew lucky ones looks efficient. With a single run per configuration — the design this study replaces — the reported number is whichever outcome that one initialisation produced, and nothing in the energy data distinguishes the two cases. This is why we report accuracy alongside energy rather than assuming a fixed budget delivers fixed quality.

RQ1 Findings: Impact of Ecosystems on Training Energy Efficiency

Measured at a common board-and-package boundary and replicated 5 times per configuration, training energy differs across ecosystems by 7.8× to 18.2× depending on architecture and dataset, with JAX cheapest in every block. Between-run variability is small (median CV 0.44 %), so these differences are well resolved. On Fashion-MNIST final accuracy converges to within 1.3 percentage points on either architecture, so there the differences are energetic rather than qualitative. On the harder datasets the stacks do *not* reach the same accuracy — 5.1 points on CIFAR-100 and 3.9 on Tiny ImageNet even among the runs that trained — and there the energy comparison must be read alongside the accuracy reached.

6.3. RQ2: Impact of ecosystems on inference energy efficiency

Table 9 reports inference energy per epoch over the test set, at the same boundary.

The inference spread is wider than the training spread (10.6× to 54.0×), and this is where the apparatus matters most. An inference pass at these dataset sizes takes well under a second in the fastest stacks, which places it in the regime where the reported window carries the constant characterised in Section 6.5. Any inference comparison drawn from an estimator's own duration field is therefore comparing padding rather than phases for exactly the stacks that are fastest. Our figures are counter-bracketed and do not have this problem, but the point generalises: the inference phase is where a cross-ecosystem energy study is most exposed to its instrument, and it is also the phase practitioners deploy.

RQ2 Findings: Impact of Ecosystems on Inference Energy Efficiency

Inference energy spreads more widely across ecosystems than training energy (10.6×–54.0×), and does so on blocks short enough that the measurement instrument, not the ecosystem, becomes the dominant source of error in a naive design. Reported inference rankings should be treated as claims about the apparatus until the window over which energy was accumulated is stated.

6.4. RQ3: Energy–time relationship across ecosystems

The claim that "faster is not greener" is a claim about two measurements, and it is only as good as the weaker of them. We therefore recompute it on counter-bracketed durations — the interval between the two counter reads that bracket a phase — rather than on any duration reported by the estimator.

Table 9
Mean inference energy per evaluation pass (J), accelerator plus CPU package.

Ecosystem	ResNet-18			VGG-16		
	CIFAR-100	F-MNIST	Tiny-IN	CIFAR-100	F-MNIST	Tiny-IN
JAX	60	44	134	112	104	222
TensorFlow	65	52	140	127	128	238
C++	73	51	136	181	177	244
Rust	101	85	350	293	276	569
PyTorch	208	231	357	344	342	454
Java	546	542	571	1 091	1 087	1 096
R	2 160	2 397	2 393	2 110	2 361	2 364

On that basis, energy and time rank the configurations almost identically: Spearman $\rho = 0.98$ for training and 0.98 for inference, with 5.1 % and 5.0 % of configuration pairs discordant respectively. Within this workload and platform, execution time is a good proxy for energy, and the interesting question is why an analysis of the same experimental shape can conclude otherwise.

The answer is in Figure 5. Panel (a) plots the duration CodeCarbon reports for each block against the counter-bracketed phase. The excess between them is not continuous: it falls into 3 tight modes and nothing lies between them. 33 % of blocks carry an excess of 0.01 s — that is, none; 49 % carry 3.27 s; and 18 % carry 4.56 s. The widest of the three modes spans 0.80 s. Two thirds of the campaign's blocks are therefore reported with a window that is seconds longer than the interval over which their energy was accumulated.

What decides which mode a block lands in is not its length. Short blocks are padded far more often than long ones, and it is tempting — we were tempted — to write the excess as a function of phase duration. It is not one. 450 blocks longer than 11 s are padded and 86 blocks shorter than it are not; more decisively, R carries no padding in any of its 1800 blocks despite having the longest blocks in the campaign, while C++ is padded in every one of its own. The excess is a property of the tracker's lifetime inside a particular process, not of the phase it brackets.

We are explicit about this because we fitted two length-based models before looking at the modes, and both are wrong in the same way. A floor, max(phase, 3.99 s), reaches $R^2 = 0.9926$. A threshold, phase plus 3.27 s below 11 s, reaches $R^2 = 0.9980$ at a third of the error, and looks conclusive. Both are curves fitted to a predictor that does not govern the phenomenon, and both achieve their R^2 against a variable whose range is dominated by phase length itself. A high coefficient of determination on a skewed predictor is not evidence of the right functional form — which is this paper's own argument, turned on this paper's own analysis, twice.

The consequence needs no model at all, which is the point. Table 10 divides each instrument's own energy by its own duration. Below half a second the estimator's fields give 21 W where the counters measure 208 W, a factor of 11.2. The distortion falls monotonically with block length and reaches unity above ten seconds. The shortest phase in the campaign takes 0.20 s and is filed as 3.47 s, a factor of 17.

Table 10
Power derived from the estimator's own energy and duration fields against power from the counters, by phase length. No model is fitted: each instrument is divided by itself.

Phase length	Blocks	Power from the reported fields	Measured power	Understated by
<0.5 s	1614	21 W	208 W	11.20×
0.5–1 s	2260	52 W	321 W	5.22×
1–2 s	794	96 W	308 W	3.02×
2–5 s	1912	157 W	306 W	2.21×
5–10 s	1195	252 W	388 W	1.32×
10–30 s	2875	345 W	388 W	0.95×
>30 s	1950	266 W	246 W	0.93×

Panel (b) shows the consequence. Power computed as reported energy over reported duration is understated by 11.2× on the shortest blocks (21 W against a measured 208 W) and is correct above ten seconds. The bias is not random and it is not uniform: it is monotone in phase length, so it falls hardest on the fastest ecosystems and on the inference phase, and it stretches their apparent time axis while leaving their energy untouched. That is precisely the shape of an artefactual "fast but energy-hungry" finding.

RQ3 Findings: Energy and Time

On counter-bracketed durations, energy and execution time rank ecosystems almost identically ($\rho = 0.98$ training, 0.98 inference): within this workload, faster *is* greener. The contrary finding is reproducible as an instrument artefact — the duration field of a widely used software estimator pads 67 % of blocks with seconds of tracker lifetime that carry no energy, inflating the apparent runtime of the shortest block by 17× while leaving its energy correct.

6.5. RQ4: How much of this is the apparatus?

Every block in the campaign carries two independent energy readings over the same window. Figure 6 and Table 11 report what they say about each other.

On energy, the two readings agree — and it is important to say why. Over the whole campaign the ratio of CodeCarbon to counters is 1.0043 for the accelerator and 1.0070 for the CPU package, or 1.0048 for their sum, a mean per-block disagreement of 0.5 %; weighted by energy over the campaign it is 0.04 %.

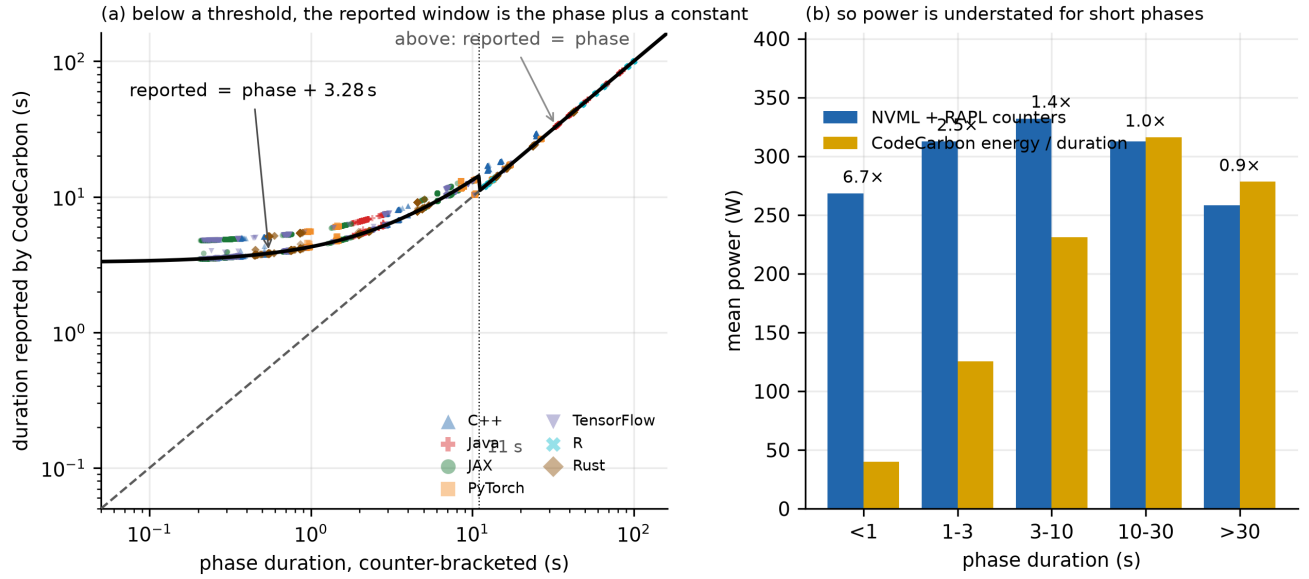

Figure 5: (a) The duration reported alongside each energy figure is not the interval the energy was accumulated over: the excess falls into 3 discrete modes, and which one a block lands in is not determined by its length. (b) Power obtained by dividing reported energy by reported duration is consequently understated for short phases and correct for long ones.

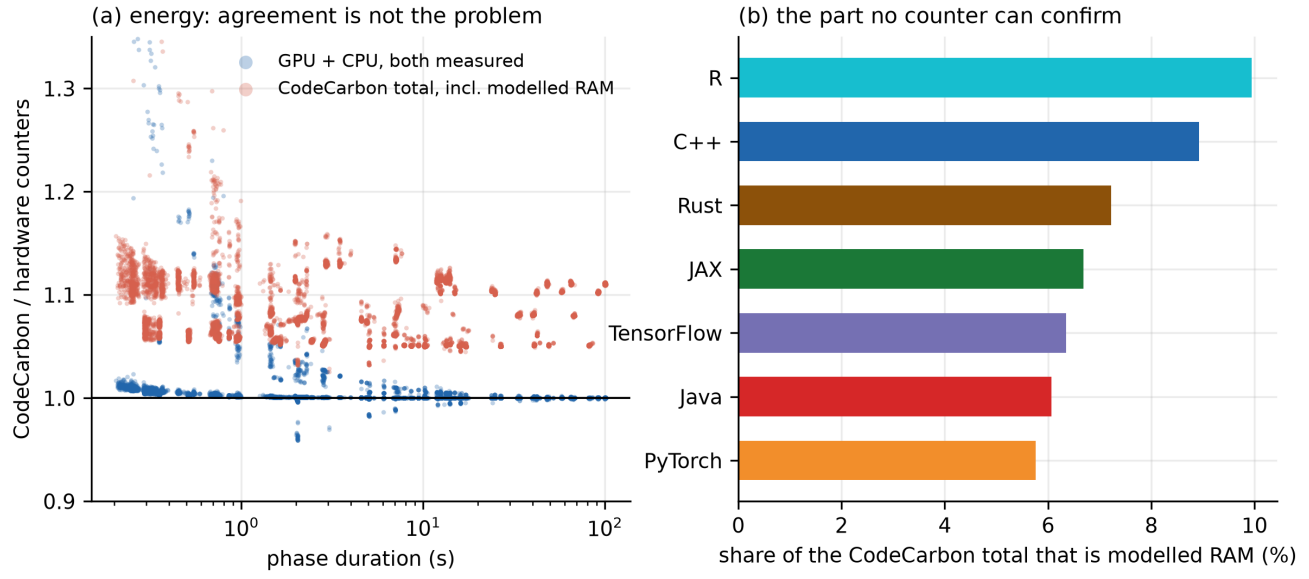

Figure 6: (a) Ratio of CodeCarbon to hardware counters, per block, against phase duration. The measured part agrees; the total does not, by exactly the modelled RAM term. (b) The share of each ecosystem's reported total that is modelled RAM.

We initially read that as an accuracy result and it is not one. Where both counters are exposed, CodeCarbon reads *the same registers we do*: NVML's accumulated-energy register on the accelerator and the RAPL `energy_uj` files on the CPU package. The two are not independent instruments; they are two readers of one instrument. The evidence is in the residual itself. The accelerator terms differ by a median of 1.3 mJ — floating-point noise on the same integer register — and essentially all of the residual is the CPU term, a near-constant 0.50 J per block whose correlation with block duration is -0.02. That constant is the counter reads being nested immediately inside the tracker's own window rather than coinciding with it: at this machine's package power it corresponds to a window a few milliseconds wider, and it does not grow with the block.

This changes what the comparison is worth, and we think it makes it more useful rather than less. It does not certify the estimator's accuracy against ground truth, which would need a wall meter and which independent validation work supplies [35]. What it certifies is that on a platform where the counters are exposed the estimator's energy is not a model at all — so the failure modes worth worrying about are not in the arithmetic but in the configurations where the counters are *not* exposed and the substitution is silent, and in the fields

Table 11
Agreement between the two readings over 12600 blocks. The mean is over per-block ratios and is dominated by the short blocks, which carry a fixed offset; the campaign-weighted column is the ratio of the two totals. The first three rows compare registers with themselves.

Quantity	Mean per block	5th–95th pct	Campaign-weighted
GPU, ratio (both read the NVML energy register)	1.004	1.000–1.006	1.000
CPU package, ratio (both read RAPL energy_uj)	1.007	1.000–1.029	1.001
GPU + CPU, ratio (the part both read)	1.005	1.000–1.011	1.000
CodeCarbon total over counters, ratio (incl. modelled RAM)	1.084	1.050–1.130	1.077
RAM share of the CodeCarbon total (per cent, not a ratio)	7.280	4.790–10.517	7.087

Table 12
Tracked share of each run's wall time, and how much of the remainder is the instrument's own window. Coverage follows block length, not the ecosystem.

Ecosystem	Median block	Tracked share	Of the untracked rest, the instrument
JAX	3.2 s	44%	97%
TensorFlow	4.2 s	57%	100%
C++	5.2 s	62%	100%
PyTorch	7.8 s	78%	99%
Rust	8.0 s	80%	100%
Java	23.8 s	93%	99%
R	37.3 s	100%	15%

beside the energy. Both are what the rest of this section is about.

It also means one figure we reported must be read carefully. Expressed as a percentage, a fixed 0.52 J offset grows without bound as blocks shorten: the mean per-block error is 0.01 % above 30 s and 1.3 % below one second. That is arithmetic on a constant, not a degradation of the instrument, and we withdraw the reading — which we published in an earlier draft of this analysis — that the transition marks the sampling interval. There is no transition: the absolute residual is flat across every duration decile from 0.3 s to 65 s.

On what is not measured, they cannot agree. CodeCarbon's total exceeds the counters by 1.084×, and the excess is entirely its RAM term — 7.3 % of its reported total on average — which is derived from installed memory capacity rather than from memory activity. The model is specific: a constant watts-per-DIMM, over a DIMM count inferred from total capacity, with decreasing marginal power above four DIMMs and a cap. On this machine it yields a flat 20 W for every block of the campaign, regardless of what the workload does to memory. No counter on this platform can confirm or refute it, because there is no DRAM RAPL domain to read.

The number is not wrong so much as it is not a measurement, and it is added silently to a figure otherwise composed of measurements. On a machine with more installed RAM the model contributes more — bounded, because of the cap, but enough that the reported energy of an identical workload depends on the host's memory configuration. Our own boundary has the opposite failing and we state it plainly: the counters omit DRAM energy altogether. One figure contains an unverifiable term; the other is missing a real one. Neither is a whole-system number, and the difference between them is not an error bar.

The time between the phases belongs to the instrument. Energy is attributed only to the phases each stack brackets, and the stacks differ enormously in how much of a run that is. Between the start of a run's first block and the end of its last, tracked time ranges from 44 % (JAX) to 100 % (R): more than half of a JAX run's measured interval falls outside any measured block. We state the denominator precisely because it is not the process's wall time — start-up, dataset construction, module loading, warm-up before the first epoch and teardown lie outside it, and are excluded from both sides of the ratio.

The natural reading is that the low-coverage stacks do a great deal of work between phases and are flattered by a per-phase comparison. We pursued that reading and it does not survive. The gap preceding each block is close to the amount by which CodeCarbon's reported window exceeds the counter-bracketed phase: the two correlate at $r = 0.98$ over every block in the campaign, their medians differ by 0.23 s, and the window excess accounts for 96 % of all untracked time. That pooled correlation is largely a contrast between padded and unpadded blocks rather than a fine agreement, and we report it as such: within the padded blocks alone it is $r = 0.82$, and within the unpadded blocks $r = 0.04$. The timestamps these gaps are computed from have 1 s resolution against a median gap of 3.13 s, which is the precision this attribution has.

Coverage tracks block length rather than anything about the ecosystems (Table 12): short blocks pay the tracker's fixed cost more often, so JAX with a 3.2 s median block is penalised where R with a 37 s one is not. The untracked time is the instrument holding its window open. It would not exist in an uninstrumented run, and charging it to the ecosystems would charge them for the cost of being measured. Per-phase energy is therefore the right quantity, and the spreads reported in Section 6.2 stand as they are.

Two caveats belong with that conclusion. The attribution is not exact — for one ecosystem the instrument's share of untracked time comes out slightly above 100 %, which is the one-second timestamp resolution showing through, and Table 12 prints it unclamped rather than rounding it away. And the untracked time is not free. Across the campaign it amounts to 8.7 hours, which at this machine's measured idle draw of 63 W is 2.0 MJ — about 4 % of the energy the campaign reports, drawn from the wall and attributed to nobody. Its share grows as blocks get shorter, which is exactly where the instrument's other failure modes bite, and it is a cost anyone planning a machine-mode campaign should budget for rather than discover.

Does the instrument change the answer? We recomputed the ecosystem ranking of each block under both readings. They agree in 4 of 6 blocks, and the disagreements are confined to adjacent pairs. So the instrument does not reorder the substantive conclusions of RQ1 — provided energy, and not power or time derived from energy, is the quantity being compared.

RQ4 Findings: The Apparatus

Over 12600 doubly instrumented blocks, a widely used software estimator agrees with hardware energy counters to 0.5 % on the quantity both measure — because on this platform it is reading the same two registers, not estimating. That is the useful finding: where the counters are exposed, the estimator's energy is not the risk. The risks are the terms beside it. A modelled RAM term contributes 7.3 % of the reported total, which no counter can check. 67 % of blocks are reported with a duration seconds longer than the interval their energy was accumulated over, so power derived from those two fields is understated by up to 11.2×. Both produce plausible numbers; neither is visible from the estimator's output alone. The ecosystem ranking is nonetheless unchanged in 4 of 6 blocks.

7. Discussion

Our study highlights that the energy efficiency of DL workloads is not only a matter of algorithmic or hardware design, but is also materially influenced by the *choice of language–framework ecosystem* (i.e. the adopted stack, including frameworks/libraries, runtimes, and toolchains) [26, 27, 18, 21]. In this section, we interpret the results in light of their broader implications, and discuss lessons learned for different stakeholders.

7.1. Interpretation of Results

The ecosystem effect is large, and it is not about the kernels. Under a common measurement boundary, the same architecture trained on the same data for the same number of epochs costs between 7.8× and 18.2× more energy on one ecosystem than another. Three of the stacks in that comparison execute a byte-identical TorchScript module, so whatever separates them is not kernel quality: it is the binding, the data pipeline, and how much host-side work each stack does per unit of accelerator work. That is an encouraging finding, because host-side overhead is the kind of thing an ecosystem maintainer can fix.

Ecosystem-level overheads amplify with workload scale. The absolute gap widens with dataset size and with model depth, so the practical relevance of the choice grows exactly where energy matters most. The mechanism cannot be isolated in a black-box study — runtime layers, input pipelines, execution engines and toolchains all differ — and we therefore report these as ecosystem-level effects rather than attributing them to a programming language.

Rankings are largely phase-consistent, contrary to what an estimator-based analysis suggests. Recomputed on counter-bracketed energy, the same ecosystem is cheapest in both phases in 6 of 6 blocks, and the full ordering is identical in 2, with Spearman correlations between 0.93 and 1.00. This matters because the opposite claim — that the phases rank differently and so demand phase-aware stack selection — is a natural conclusion to draw from estimator output, and we can now say why: inference blocks in this workload are short enough to fall in the regime where the reported window carries the constant of Section 6.5, so an inference ranking derived from an estimator's own duration is partly a ranking of that constant. We do not claim phase-dependence never occurs; we claim that establishing it requires an instrument whose window is known.

Faster *is* greener, here. On measured durations, energy and time rank configurations at $\rho = 0.98$ (training) and 0.98 (inference). This contradicts a well-established finding in cross-language energy work [30, 32, 18], and we want to be careful about the scope of the contradiction. Those studies largely concern CPU-bound general-purpose benchmarks, where a language can trade instruction count against memory traffic and shift the energy-per-second substantially. In a GPU-bound DL workload the accelerator dominates and runs near a narrow band of power, so time and energy are nearly proportional almost by construction. The interesting result is not that we disagree, but that a study of this exact shape can produce either answer depending on which of two fields of the same CSV it treats as the duration.

Quality must be measured, not assumed. Under a fixed epoch budget the stacks converge to within 1.3 percentage points on Fashion-MNIST but separate by up to 7.5 points per block on the harder datasets, counting only the runs that trained. Any efficiency claim on those datasets that does not report accuracy alongside energy is comparing stacks that did different amounts of learning. This is not a subtle correction: an ecosystem that converges more slowly looks efficient precisely because it accomplished less. We stress "counting only the runs that trained", because the uncorrected spread is 13.1 points on CIFAR-100 and most of that is the collapses of Section 6.2.1 rather than any difference in learning.

7.2. Implications for Stakeholders

Our findings carry different implications for different communities:

- **Researchers:** energy should be reported alongside accuracy, and both should be reported alongside the measurement boundary. Our stronger recommendation is procedural: a cross-stack study needs a written specification and a mechanism that enforces it. We found that every divergence we eventually corrected had produced a plausible number first, and that inspection did not catch them — executable checks did.

- **Practitioners:** ecosystem choice has a first-order effect on energy, and in this workload the same choice serves both phases, so the "one stack for training, another for serving" complexity is not obviously warranted. Efficiency must nonetheless be weighed against maintainability, hiring, library availability and time to delivery; an energy gap of this size is not a mandate when the cheaper stack has a tenth of the ecosystem around it.

- **Framework designers:** efficiency stems from the ecosystem as a whole, and the largest recoverable gaps we observe are in host-side work — data

loading, dispatch and binding overhead — rather than in kernels. We would also ask for one specific thing from measurement tooling: report the interval over which energy was accumulated, and fail rather than substitute a model when a counter is unavailable. An instrument that refuses to run is more useful than one that quietly estimates.

- **Policy makers and funding agencies:** AI efficiency is multi-dimensional, and we caution against adopting any single metric as a target (Goodhart's Law). Reporting requirements should ask for the measurement boundary and the replication count, not only the number: a figure without either is not auditable, and our own experience is that unauditable figures are wrong more often than not.

7.3. AI Requires Explicit Energy Measurement

Ecosystem selection is not a neutral engineering choice: it shifts DL energy consumption by a factor that is large, reproducible and, in the case of host-side overhead, addressable. That is the substantive result.

The methodological result is that establishing it is harder than it looks. Every defect we found in building this study produced numbers of the right order of magnitude, in the right units, with plausible relative ordering. A learning rate that differs between two framework APIs, an evaluation loop written one image at a time, a linker dropping a CUDA dependency so that a binary runs on the CPU, an estimator reporting a padded window where a duration belongs — none of these announce themselves in the data. They are detectable only by a second instrument, an executable specification, or a quality metric that the study happened to record.

We therefore make the artefacts as important as the findings. The specification, the shared measurement contract, the conformance checker and the raw dual-instrument records are all in the replication package, and the manuscript's numbers are generated from them rather than transcribed. A reader who doubts a figure in this paper can regenerate it; a reader who doubts the apparatus can run the checks.

7.4. An Industrial Simulation of Deep Green AI Large-Scale Impact

To contextualize our empirical results in a hyperscale setting, we present an illustrative industrial scenario that translates our *relative* ecosystem-level energy differences into approximate energy and cost impacts. This scenario is not intended as a direct estimate for any specific production system, model family, or hardware stack; rather, it shows how small per-workload gaps can compound at scale.

Relative multipliers. For a given phase (training or inference) and ecosystem k, we define a phase-specific energy multiplier relative to any reference ecosystem *base* as:

$$m_k = \frac{\bar{E}_k}{\bar{E}_{\text{base}}},$$

where $\bar{E}_k$ is the mean energy measured under our experimental protocol for that phase. Accordingly, m_k should be interpreted as a *relative energy-per-work* factor under our setup, not as a universal constant.

Under the simplifying assumption that the ecosystem-level multiplier transfers to the same workload definition, for any baseline energy budget $\bar{E}_{\text{base}}$, the scaled energy is calculated as $\bar{E}_k = m_k \cdot \bar{E}_{\text{base}}$.

Training phase (illustrative scaling). From Table 4, moving from the most expensive ecosystem in a block to the cheapest changes training energy by 7.8×–18.2×. Taking a concrete case: if a *training* budget is $E = 1$ GWh on the most expensive stack of a block, the same fixed-budget run costs between $1/18.2$ and $1/7.8$ of that on the cheapest. At an electricity price of 100 €/MWh — a round figure in the range of recent European non-household prices, chosen for legibility rather than precision — the spread between the two ends of a single block is on the order of hundreds of thousands of euro per GWh of training. We stress that this is arithmetic on a relative multiplier, not a forecast: it assumes the multiplier transfers to a workload of a different size on different hardware, which is exactly the assumption our own measurements give us the least confidence in.

Inference phase (hyperscale workload, fixed work). Let PyTorch serve a fixed daily inference volume on $N = 100{,}000$ accelerators running continuously for 24 hours at an average inference-attributable draw of $P = 0.4$ kW each: 960 MWh/day, or 96 000 € per day at 100 €/MWh.

Two things about how that scales. First, the quantity a less efficient stack needs more of is *accelerator-hours*, not watts: at a fixed service target it needs a larger fleet, or the same fleet for longer, and Table 13 states it that way. Multiplying energy at a fixed fleet and fixed hours would imply a per-card draw several times the board limit of the accelerator we measured, which is not a thing that can happen. Second, the multipliers are computed from *accelerator energy alone*, to match the per-accelerator power budget the scenario assumes; taking them at the board-and-package boundary instead moves the ecosystem spread from 18.4× to 20.4×. This matters because the submitted version of this paper multiplied a per-GPU board power by a ratio derived from CPU, GPU and modelled RAM energy together — different boundaries, and their product is not a quantity.

On that basis JAX serves the same volume at 0.34× the baseline energy and R at 6.19×. We note that the inference blocks these multipliers come from are sub-second in most stacks, which is the regime Section 6.5 identifies as least reliable for any instrument; they are counter-bracketed and so do not carry the estimator's padding, but they are the shortest measurements in the campaign and the ratios inherit that.

Takeaway. Ecosystem-level energy differences, treated purely as relative multipliers under a uniform protocol, translate into large operational deltas at hyperscale. Two caveats bound that statement, and both come from this study rather than from convention. The multipliers were measured on a single-accelerator desktop at 32 × 32 input resolution, a

Table 13
Projected daily energy and cost for large-scale inference (10^5 accelerators, 24 h, electricity at 100 €/MWh), scaling the *relative* inference energy measured in this campaign with Python/PyTorch as the baseline. These are scenario arithmetic on measured multipliers, not forecasts.

Ecosystem	Relative energy	Accelerators	Energy (MWh/day)	Cost (€/day)
JAX	0.34×	33 645	323	32 299
TensorFlow	0.38×	37 995	365	36 475
C++	0.44×	44 138	424	42 372
Rust	0.85×	85 466	820	82 047
PyTorch (baseline)	1.00×	100 000	960	96 000
Java	2.67×	266 774	2 561	256 103
R	6.19×	618 858	5 941	594 104

regime in which data movement and dispatch weigh more heavily than they would at production scale, so they are more likely to overstate binding overhead than to understate it. And the whole calculation is a scaling of *relative* differences: had we reported the modelled component of a software estimator's output as if it were measured, the same arithmetic would have produced a confident number from a quantity that depends on how much memory the host happens to have.

8. A catalogue of defect classes

Building this study meant first auditing an earlier campaign of the same design and then repairing every stack against the specification. Table 14 collects what that turned up. We report the defects as *classes* rather than as a list of bugs in one project, because each is a plausible mistake in any study of this shape, several are language or tool *defaults* rather than oversights, and most are invisible in the resulting measurements.

The last column is the one that matters. It asks whether a reader given only the released data — the tables, the CSVs, the plots — could detect the problem.

One row of that table deserves to be made concrete, because it is the class most likely to affect any study of this shape and the least likely to be noticed. Table 15 reports the training protocol *as implemented*, read from the source of an earlier campaign whose manuscript stated a single common learning rate.

Two stacks train at ten times the stated learning rate. Loader parallelism ranges from none to every hardware thread on the machine, and across the campaign it correlates with epoch duration at $\rho = -0.73$ ($p = 0.041$) over a 11.6× spread in duration and 9.9× in energy — which is to say that a column no one would think to report tracks the published ranking better than the programming language does. One stack normalises its inputs when no other does, so it is not even computing the same function. None of this is visible in an energy figure, and all of it is verifiable in about a page of code.

Three observations follow.

Only two of these are visible from the released data — and it is the one that changes no conclusion, since a constant factor leaves every ranking intact. Everything that does change a conclusion requires reading the code, or a second instrument, or a quality metric the study has to have thought to record.

Several are defaults rather than oversights. The linker's `-as-needed` behaviour, an estimator's fallback power constants, a framework resolving CUDA wheels that do not match its build, a resize function that returns `uint8` because the images it was written for were `uint8`. None is a lapse in care. A study can be executed carefully and land on all of them.

Some of them are ours. 5 of the 23 entries were introduced while repairing the study, and each was caught by a mechanism rather than by attention: the conformance checks, the accuracy metric, the second instrument, or the campaign driver refusing to start.

One of them deserves singling out, because it is not a mistake in the usual sense. Moving the input transform into the loader is required by S3. Switching evaluation to batched inference is required by S3 as well. Each change is correct, and each was reviewed. Together they left a private copy of the transform on the training path and removed it from the evaluation path, and the result trained a network on black images for thirty epochs while reporting a plausible falling loss. No amount of care applied to either change in isolation would have caught it; what caught it was a metric neither change was about. We report them because the alternative — a paper claiming its authors made no mistakes — would undercut the argument it is making. The argument is that enforcement beats discipline, and the evidence for it is that our own discipline was not sufficient either.

9. Threats to Validity

As with any empirical study, our work is subject to threats to validity. We group them conventionally, and note where a threat that would ordinarily be speculative is one we can quantify from the data.

Internal Validity. Whole-machine energy tracking means any concurrent work contaminates a measurement. The host was otherwise idle for the duration of the campaign; the blocks measured during development while a package download was running were discarded rather than corrected, and the protocol amended to forbid it. We report this because it is an easy mistake to make and an invisible one to inherit.

Table 14
Defect classes, with the measured effect of each. "Visible" asks whether a reader given only the released measurements could detect the problem. Entries marked ∗ were introduced by us while repairing the study and caught by the checks or the accuracy metric, not by review.

Class	Measured effect	Visible
Instrument		
Unit mislabelling	Energy reported in kW h under a J label; a factor of 3.6×10^6	yes
Reported duration is not the accumulation window	67 % of blocks padded by seconds of tracker lifetime in 3 discrete modes; power understated 11.2× on the shortest blocks, correct above ten seconds	no
Modelled term inside a measured total	RAM energy from installed capacity, 7.3 % of the reported total; scales with the host, not the workload	no
Mixed instrument versions	Versions 2.8.4 and 3.0.4 in one campaign: 231 W against 102 W of modelled host power for the same machine	no
Mixed tracking mode	1 of 8 stacks tracked per-process rather than machine-wide; its RAM term is 0.026 % of reported energy against 25 % to 55 % for the rest	no
Interpreter resolved from `PATH`	The instrument's version becomes a property of the shell; 5 of 8 stacks report a single constant CPU power, the signature of a modelled fallback	no
Unsynchronised tracker	Inference recorded as 0 J; training understated 24 %	no
Host not idle	Whole-machine tracking charges unrelated downloads to the workload	no
Two campaigns on one accelerator∗	An orphaned driver restarted and wrote the same run directories while sharing the GPU: runs of 59, 60 and 43 epochs where the specification says 30, each measured against the other's contention	yes
Implementation		
Divergent hyperparameters	2 learning rates across 8 stacks	no
Divergent loader parallelism	0 to 96 threads; $\rho = -0.73$ against epoch duration ($p = 0.041$) over a 11.6× spread	no
Divergent input shape	One stack at 0.26× another's `conv1` input volume; 0.07× the energy	no
Divergent evaluation batching	Batch 1 against 128; 1.5–1.95× inference energy	no
Evaluation in training mode	Test accuracy 21 % against 86 %; a different computation measured	no†
Resize discards a dtype conversion∗	Image scaled to $[0, 1]$, then resized by a function returning `uint8`; the scaling is silently thrown away	no‡
Two correct changes that are wrong together∗	Moving the transform into the loader, and batching evaluation, each right alone; together they left a private copy of the transform on the training path only, which trained the network on black images for thirty epochs	no†
Channel order assumed, not checked∗	`[C,H,W]` treated as `[H,W,C]`: the width cropped to three pixels and the result transposed	no‡
Ambiguous dataset layout	A class name containing / yields three different datasets	no
Placement		
Silent CPU fallback	Whole workload on the CPU after a CUDA wheel mismatch; one warning line	no
Linker drops the CUDA dependency	Same source, same LibTorch: `Cpu` against `Cuda(0)`	no
Analysis		
Collapsed runs counted as measurements	12 runs at chance accuracy averaged in with runs that trained; 20.2 against 1.3 points of apparent cross-stack spread	no†
Partial runs averaged as measurements∗	A crashed run's fragment, 16 J against a neighbour's 300 kJ, produced ecosystem spreads of 2×10^4	no
Epochs treated as replications	Effective sample size 1 per configuration; intervals far too narrow	no

† invisible because no stack wrote its accuracy to disk; visible the moment one does.
‡ invisible in energy data; here it happened to abort the run, but the channel-order defect would have trained quietly on a three-pixel sliver had the dtype defect not stopped it first.

Toolchain heterogeneity remains. CUDA versions across the 7 stacks range from 11.6 (Deeplearning4j, whose final release predates CUDA 12) to 12.8 (the pinned LibTorch shared by four ecosystems). This is not fully controlled and cannot be: an ecosystem is partly defined by the toolchain it supports. What we have changed is that it is now *bounded*. The four LibTorch stacks are pinned to one build, so the control group of Table 6 isolates binding overhead from backend differences, and a reader can see how much of the total spread survives that isolation.

Table 15
Training protocol as implemented in an earlier campaign of this design, read from source rather than from the manuscript. The manuscript stated one learning rate and one input scaling; the code contains two of each. Divergences in bold. Specification S2 and S3 now fix every column, and the conformance checks verify them against the source.

Ecosystem	Optimiser	LR	Loader mechanism	Threads	Input scaling
Python/PyTorch	Adam	1e-4	DataLoader workers	2	[0, 1]
Python/TensorFlow	Adam	**1e-3**	Keras generator	1	[0, 1]
Python/JAX	Adam	1e-4	Keras generator	1	[0, 1]
C++/LibTorch	Adam	1e-4	DataLoader workers	2	[0, 1]
Java/DL4J	Adam	1e-4	AsyncDataSetIterator	2	[0, 1]
R/torch	Adam	1e-4	dataloader workers	**0**	[0, 1]
Rust/tch	Adam	**1e-3**	rayon, all cores	**96**	**mean/std**
MATLAB/DLT	adam	1e-4	augmentedImageDatastore	1	[0, 1]

Implementation drift between stacks was the dominant internal threat in studies of this shape, and it is the one this design attacks directly. Under specification S1 the LibTorch-based ecosystems train the same exported module rather than four reimplementations; S2 and S3 fix the optimisation and the data pipeline; and 67 automated checks verify all of it against the source. 5 defects introduced during development were caught by a mechanism — a check, the accuracy metric, the second instrument, or the driver refusing to start — rather than by review. We do not claim the specification is complete — only that a divergence outside it now has to survive an executable check rather than an author's attention.

Accelerator placement was verified rather than assumed. Every harness asserts the device at startup and refuses to fall back to the CPU. This is not hypothetical: linking a Rust binary against a CUDA LibTorch produces, by default, a binary that runs entirely on the CPU, because `-as-needed` drops the `torch_cuda` dependency that no Rust symbol references. Nothing in the energy data distinguishes that case from a correct one except its magnitude.

Construct Validity. Our construct is energy at a stated boundary, not "sustainability". We report accelerator-board plus CPU-package energy read from hardware counters, which excludes the power supply, fans, drives, chipset — and host DRAM, for which this platform exposes no counter. We had no wall meter and therefore do not report a whole-system figure, and a board-plus-package number must not be compared with a wall-meter number from another study.

The counters are our reference and they are not exempt from the scrutiny we apply to the estimator. RAPL's package energy is itself a modelled quantity on some parts and its update interval is of order a millisecond [54]; NVML's accumulated-energy register rests on board telemetry averaged internally, so its temporal resolution is not unlimited either. That bounds what the sub-second comparison of Section 6.5 can establish: for the shortest blocks in this campaign, both instruments are near the edge of what they resolve, and the disagreement we report there should be read as a joint property of the pair rather than as a fault of one. It does not bear on the reported-duration finding, which concerns a field rather than a measurement.

A second construct limitation belongs here, because it bounds what the attribution in Section 7 can claim. Every figure we report is whole-machine and un-baselined: the counters accumulate whatever the silicon draws, the machine's static consumption included. We measured that directly rather than arguing about it. On an idle host at the same boundary this machine draws 63 W — 24 W at the accelerator and 39 W at the CPU package — which is 21 % of the mean power of a measured block.

The two halves of the boundary are affected very differently, and this is the part that matters for interpretation. The accelerator's idle draw is 10 % of its campaign mean, so the GPU term is mostly workload. The CPU package's is 71 %: of the 54 W mean package power, the great majority is the machine being on. That term accordingly carries almost no workload signal — its coefficient of variation across the whole campaign is 6 % against 35 % for the accelerator — while contributing 19 % of reported energy. So when we locate the ecosystem spread in host-side work, that is an inference from accelerator idle time and from the shared-module control group, not from a measured host-side energy term. A study wanting to measure host-side work directly would need DRAM and platform counters this machine does not expose. We report un-baselined energy because that is what the counters give and what an operator pays for, and we state the baseline so a reader can subtract it.

The more interesting construct threat is one we can measure. A software estimator's output is a mixture of measured and modelled terms, and the mixture is not stated in the output. Over 12600 blocks, CodeCarbon agrees with the counters to 0.5 % on the terms both measure, while its total exceeds them by 1.084× — the difference being a RAM term derived from installed capacity, 7.3 % of the reported total here and necessarily larger on a large-memory host. A figure of that construction corresponds to no physical boundary, and its value depends on the host's memory configuration rather than on the workload.

The estimator's reported duration is a second construct threat, and the sharper of the two: below about 11 s it is the phase plus a constant 3.27 s rather than the accumulation window, so any power or energy–time quantity derived from it is biased as a function of phase length. We use counter-bracketed durations throughout for this reason.

External Validity. We studied two convolutional architectures and three image classification datasets at 32×32 resolution; results may not generalise to other model families, to transformers, or to higher-resolution workloads, where the balance between accelerator work and host-side work is different — and, given that host-side work is where we locate most of the observed spread, likely more favourable to the slower ecosystems than what we report.

The workload also does not saturate the accelerator. At this resolution and these dataset sizes the GPU runs well below its board power limit for much of each epoch, so the campaign measures data loading and dispatch more than it measures deep learning. This bounds the generality of the absolute figures in a specific direction: a saturating workload would compress the ecosystem spread.

The platform is a single-accelerator desktop rather than a datacentre node, which makes the host share of total energy smaller than it would be on a multi-socket server. And MATLAB, present in an earlier campaign of this design, is out of scope here because it cannot be brought into conformance with the specification; readers should not read its absence as a result about MATLAB.

Conclusion Validity. The submitted analyses of studies of this shape — including an earlier one of our own — treat the epochs of a single run as repeated measurements. They are not independent: epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, so the effective sample size per configuration is one, confidence intervals are far too narrow and significance is inflated. Every interval and test reported here is computed on 5 independent runs per configuration, and 185 of the 210 runs were interleaved rather than executed consecutively, so that machine-state drift is spread across conditions instead of aliasing onto one ecosystem. The exception is stated at the end of this subsection.

Five repetitions is a small sample for a rank test, and we report what that costs rather than working around it. A Kruskal–Wallis test rejects equality of ecosystems in every block ($p < 10^{-4}$, $\varepsilon^2 \geq 0.95$), and 99 % of the 252 pairwise comparisons carry a large Cliff's delta. Yet 0 of those 252 pairs is significant after Holm correction — and cannot be, at any effect size. With five runs against five, the smallest attainable two-sided exact Mann–Whitney p is 0.0079, and Holm over the 21 comparisons within a block multiplies it to 0.167. The design has the power to establish that the ecosystems differ and not the power to order any particular pair; we therefore rest the ordering claims on effect sizes and intervals, and make no pairwise significance claim.

Between-run variability is low — a median coefficient of variation of 0.44 % for training energy — which makes the design well powered for the effect sizes we report. Low variance is a statement about the measurement rather than a guarantee that the measurement is of the right thing.

Where the sample is genuinely too small we say so rather than reach. The VGG-16 collapse rates of Section 6.2.1 range from 0 % to 50 % across ecosystems, and a χ^2 test on that table returns $p = 0.0065$, which would license a claim that ecosystems differ in robustness; a permutation test we ran on the same data returns $p = 0.07$, which would not.

We report both, and we are careful about why they differ, because our first explanation was wrong. It is tempting to attribute the gap to the χ^2 approximation failing on small cells — the smallest expected count here is 1.7. That is not what happened. The two tests use different statistics: our permutation test calibrates the *range* of per-ecosystem rates, a bounded statistic that saturates once one ecosystem sits at zero and another at a half, and it is much the less powerful of the two. Permutation-calibrating the χ^2 statistic itself reproduces the χ^2 p-value closely. The honest reading is that the evidence does lean towards ecosystem-dependent collapse rates, on ten susceptible runs per ecosystem, and that ten runs cannot settle it. We make no claim about which ecosystems are more robust, and we flag the question as one this design was not built to answer.

Finally, 25 runs — 5 configurations, all of them Rust — were executed in a separate window 5 days after the rest of the campaign, once defects in that stack's image loaders were corrected. They cover ResNet-18/Fashion-MNIST, ResNet-18/Tiny ImageNet, VGG-16/CIFAR-100, VGG-16/Fashion-MNIST, VGG-16/Tiny ImageNet. They are therefore not interleaved with the other ecosystems, and machine-state drift between the two windows is not controlled for them. Given the between-run coefficients of variation we observe, we expect this to be small relative to the effects reported, but it is a real asymmetry in an otherwise interleaved design and we flag it rather than absorb it.

10. Conclusion

This paper presented an empirical study of how *language–framework ecosystems* affect the energy efficiency of *Deep Learning* workloads, across 7 ecosystems, two convolutional architectures and three datasets, replicated 5 times per configuration and instrumented twice per measurement block.

Three findings stand out.

First, ecosystem selection substantially shifts operational efficiency: at a common board-and-package boundary, training the same model on the same data costs between 7.8× and 18.2× more energy on one ecosystem than another, with between-run variability of order 0.44 %. Because three of the ecosystems compared execute a byte-identical TorchScript module, this gap is not kernel quality: it is binding overhead, data movement and host-side dispatch — which is to say it is addressable by the people who maintain these stacks.

Second, on the easiest dataset the effect is energetic and not qualitative: all 7 stacks converge to within 1.3 percentage points on Fashion-MNIST under the same epoch budget, doing the same work at very different cost. That equivalence does not extend to the harder datasets, where the stacks differ by 5.1 and 3.9 points even after excluding the runs that failed to train, and where energy must therefore be read alongside accuracy rather than instead of it.

Third — and this is the finding we did not expect — the apparatus determines more of the answer than the ecosystems do, in one specific and correctable way. Over 12600 doubly instrumented blocks, a widely used software estimator agrees with hardware energy counters to 0.5 % on energy. But the duration it reports beside that energy is not the interval the energy was accumulated over: 67 % of blocks carry seconds of tracker lifetime in which no energy was drawn, so power derived from the two fields is understated by up to 11.2× on the shortest blocks and correct on the longest. Recomputed on measured durations, energy and time rank ecosystems at $\rho = 0.98$: in this workload, faster *is* greener, and the widely reported contrary result is reproducible here as an instrument artefact.

We draw a procedural conclusion from that. A cross-ecosystem energy study needs a written specification, a mechanism that enforces it, and a second instrument — for the same reason that a performance benchmark needs a stated warm-up policy. Every defect we corrected in building this study produced a plausible number first. The specification, the shared measurement contract, the 67-check conformance checker and the raw dual-instrument records are part of the replication package, and every figure in this paper is generated from them.

11. Future Work

Several directions remain open.

First, the workload should be extended beyond CNNs, to transformers and multimodal architectures, and to input resolutions that saturate the accelerator. Our workload does not: the GPU runs well below its board power limit for much of each epoch, which is precisely the regime in which host-side overhead — the component we find dominates the ecosystem gap — weighs most. Establishing how the gap narrows as the accelerator saturates is the single most useful follow-up, and it is a measurement, not a speculation.

Second, testing on heterogeneous hardware (server-class accelerators, TPUs, embedded and edge devices) would establish how much of the ranking is a property of the stacks and how much of this host. The hardware–software stack should be treated as part of the ecosystem definition rather than as a nuisance parameter.

Third, a white-box layer complementing this black-box design would localise where the host-side overhead originates: loader thread scheduling, host-to-device transfer, per-kernel launch overhead, allocator behaviour. We can say that three stacks running identical kernels differ by a large factor; we cannot yet say which line of the binding is responsible.

Fourth, the instrument characterisation invites replication. We report the reported-window padding of one estimator on one platform. Whether other estimators share it, whether it varies with sampling interval and tracking mode, and how many published energy–time results rest on it, are answerable questions, and the method — bracket every block with hardware counters and compare — is cheap.

Finally, a wall-meter reference would let the board-and-package boundary used here be related to whole-system energy, which is what an operator pays for. We report that boundary consistently and decline to extrapolate; closing that gap requires hardware we did not have.

12. Carbon Footprint of This Study

The campaign reported here consumed 48.7 MJ (13.5 kWh) at the accelerator and CPU package across 210 runs. At the grid carbon intensity CodeCarbon assigns to this location, that corresponds to a few kilograms of CO_2e — modest against industrial training, and reported here for the same reason we ask others to report it: an energy figure without a stated boundary and a stated scope is not auditable. The figure above is the energy inside the measured blocks. It excludes development, failed runs and the earlier campaign that motivated this design — including those would roughly double it — and it excludes the machine's draw during the untracked time between blocks discussed in Section 6.5, a further 2.0 MJ. An honest campaign-level figure is therefore larger than the one we report and we do not pretend otherwise; what we report is the quantity the comparisons are made on.

CRediT Authorship Contribution Statement

Leonardo Pampaloni: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Visualization. **Marco Pagliocca:** Methodology, Software, Validation, Investigation, Data curation, Writing — original draft, Visualization. **Enrico Vicario:** Resources, Supervision, Funding acquisition, Writing — review & editing. **Roberto Verdecchia:** Conceptualization, Methodology, Validation, Writing — review & editing, Supervision, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

All data and code underlying this study are public. The replication package contains the implementations for all 7 ecosystems, the shared measurement bridge, the experiment specification and its conformance checker, the campaign driver, the per-block dual-instrument records for all 210 runs and 12600 blocks, and the analysis pipeline that turns those records into every table, figure and quoted number in this manuscript. A single command regenerates the manuscript from the raw records, so any number here can be traced to the measurement it came from.

13. Declaration of Generative AI and AI-assisted Technologies in the Writing Process

During the preparation of this work the authors used generative AI assistants (ChatGPT and Claude) in three distinct capacities, which we separate because they carry different weight.

For *writing*, the tools were used to improve the readability of the manuscript text. For *software engineering*, they were used to repair and refactor the experimental implementations, to write the shared measurement bridge, the conformance checker and the analysis pipeline, and to diagnose defects in the ecosystems' data loaders and build configurations. For *analysis*, they were used to write the scripts that produce the tables and figures; the scripts, not the assistants, produce the numbers, and the numbers are injected into this manuscript from those scripts rather than transcribed.

No experimental measurement was produced, adjusted or selected by an AI assistant. The authors reviewed and edited all generated content and code, verified the findings against the raw data, and take full responsibility for the content of the article.

References

[1] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.

[2] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, "Quantifying the carbon emissions of machine learning," *arXiv preprint arXiv:1910.09700*, 2019.

[3] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?" in *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021, pp. 610–623.

[4] J. Xu, W. Zhou, Z. Fu, H. Zhou, and L. Li, "A survey on green deep learning," *arXiv preprint arXiv:2111.05193*, 2021.

[5] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021.

[6] A. S. Luccioni and A. Hernandez-Garcia, "Counting carbon: A survey of factors influencing the emissions of machine learning," *arXiv preprint arXiv:2302.08476*, 2023.

[7] L. H. Kaack, P. L. Donti, E. Strubell, G. Kamiya, F. Creutzig, and D. Rolnick, "Aligning artificial intelligence with climate change mitigation," *Nature Climate Change*, vol. 12, no. 6, pp. 518–527, 2022.

[8] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. R. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022.

[9] A. S. Luccioni, S. Viguier, and A.-L. Ligozat, "Estimating the carbon footprint of BLOOM, a 176B parameter language model," *Journal of Machine Learning Research*, vol. 24, no. 253, pp. 1–15, 2023, arXiv:2211.02001.

[10] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. Di Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong, "MLPerf Tiny benchmark," in *Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks*, 2021, arXiv:2106.07597.

[11] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020.

[12] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[13] R. Verdecchia, J. Sallou, and L. Cruz, "A systematic review of Green AI," *WIREs Data Mining and Knowledge Discovery*, vol. 13, no. 4, p. e1507, 2023.

[14] V. Mehlin, S. Schacht, and C. Lanquillon, "Towards energy-efficient deep learning: An overview of energy-efficient approaches along the deep learning lifecycle," *arXiv preprint arXiv:2303.01980*, 2023.

[15] C. E. Tripp, J. Perr-Sauer, J. Gafur, A. Nag, A. Purkayastha, S. Zisman, and E. A. Bensen, "Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations," *arXiv preprint arXiv:2403.08151*, 2024.

[16] T. Yarally, L. Cruz, D. Feitosa, J. Sallou, and A. van Deursen, "Uncovering energy-efficient practices in deep learning training: Preliminary steps towards green AI," in *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 2023, pp. 25–36, arXiv:2303.13972.

[17] L. Cruz, J. P. Fernandes, M. H. Kirkeby, S. Martínez-Fernández, J. Sallou, H. Anwar, J. Bogner, J. Castaño, F. Castor, D. Feitosa, A. González, P. Lago, H. Muccini, A. Oprescu, P. Rani, J. Saraiva, F. Sarro, R. Selvan, K. Vaidhyanathan, R. Verdecchia, and I. P. Yamshchikov, "Greening AI-enabled systems with software engineering: A research agenda for environmentally sustainable AI practices," *ACM SIGSOFT Software Engineering Notes*, 2025, arXiv:2506.01774.

[18] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva, "Ranking programming languages by energy efficiency," *Science of Computer Programming*, vol. 205, p. 102609, 2021.

[19] S. Georgiou, M. Kechagia, and D. Spinellis, "Analyzing programming languages' energy consumption: An empirical study," in *Proceedings of the 21st Pan-Hellenic Conference on Informatics (PCI)*, 2017.

[20] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. P. Fernandes, J. Saraiva, and J. Cunha, "Energy efficiency across programming languages: how do energy, time, and memory relate?" in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*, 2017, pp. 256–267.

[21] A. Gordillo, C. Calero, M. Á. Moraga, F. García, J. P. Fernandes, R. Abreu, and J. Saraiva, "Programming languages ranking based on energy measurements," *Software Quality Journal*, vol. 32, no. 4, pp. 1539–1580, 2024.

[22] T. B. Chandra, P. Verma, and A. K. Dwivedi, "Impact of programming languages on energy consumption for sorting algorithms," in *Software Engineering*, ser. Advances in Intelligent Systems and Computing, vol. 731. Springer, Singapore, 2019.

[23] S. Mahadevappa and S. Figueira, "Energy-efficient programming languages for mobile applications," in *2021 IEEE Global Humanitarian Technology Conference (GHTC)*, 2021.

[24] S. Maleki, C. Fu, A. Banotra, and Z. Zong, "Understanding the impact of object oriented programming and design patterns on energy efficiency," in *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*, 2017, pp. 1–6.

[25] S. Nanz and C. A. Furia, "A comparative study of programming languages in Rosetta Code," in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, vol. 1, 2015, pp. 778–788, arXiv:1409.0252.

[26] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, and Y. Zou, "Green AI: Do deep learning frameworks have different costs?" in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1082–1094.

[27] N. Marini, L. Pampaloni, F. Di Martino, R. Verdecchia, and E. Vicario, "Green AI: Which programming language consumes the most?" in *2025 IEEE/ACM International Workshop on Green and Sustainable Software (GREENS)*, 2025, arXiv:2501.14776.

[28] N. Alizadeh and F. Castor, "Green AI: A preliminary empirical study on energy consumption in DL models across different runtime

infrastructures," in *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering — Software Engineering for AI (CAIN)*, 2024, pp. 134–139.

[29] H. Li, G. K. Rajbahadur, and C.-P. Bezemer, "Studying the impact of TensorFlow and PyTorch bindings on machine learning software quality," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, pp. 6:1–6:31, 2024.

[30] M. Weber, C. Kaltenecker, F. Sattler, S. Apel, and N. Siegmund, "Twins or false friends? A study on energy consumption and performance of configurable software," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 2098–2110.

[31] J. Dodge, T. Prewitt, R. Tachet des Combes, E. Odmark, R. Schwartz, E. Strubell, A. S. Luccioni, N. A. Smith, N. DeCario, and W. Buchanan, "Measuring the carbon intensity of AI in cloud instances," in *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2022, pp. 1877–1894.

[32] R. Rua and J. Saraiva, "A large-scale empirical study on mobile performance: energy, run-time and memory," *Empirical Software Engineering*, vol. 29, no. 1, p. 31, 2024.

[33] S. Aquino-Brítez, P. García-Sánchez, A. Ortiz, and D. Aquino-Brítez, "Towards an energy consumption index for deep learning models: A comparative analysis of architectures, GPUs, and measurement tools," *Sensors*, vol. 25, no. 3, p. 846, 2025, compares OpenZmeter hardware sensors against CarbonTracker and CodeCarbon on AlexNet, ResNet-18, VGG-16 and Swin Transformer.

[34] L. Bouza, A. Bugeau, and L. Lannelongue, "How to estimate carbon footprint when training deep learning models? A guide and review," *Environmental Research Communications*, vol. 5, no. 11, p. 115014, 2023.

[35] R. Fischer, "Ground-truthing AI energy consumption: validating CodeCarbon against external measurements," *it - Information Technology*, vol. 67, no. 4-6, pp. 155–163, 2026, reports estimation errors of up to 40% for established software estimators against meter ground truth.

[36] B. Courty, V. Schmidt, S. Luccioni, G. Kamal, M. Coutarel, B. Feld, M. Lécauchois, K. Lottick *et al.*, "`mlco2/codecarbon`: v3.3.0," 2026. [Online]. Available: https://github.com/mlco2/codecarbon

[37] E. Paula, J. Soni, H. Upadhyay, and L. Lagos, "Comparative analysis of model compression techniques for achieving carbon efficient AI," *Scientific Reports*, vol. 15, p. 23461, 2025.

[38] T. Almog, "An analysis of optimizer choice on energy efficiency and performance in neural network training," *arXiv preprint arXiv:2509.13516*, 2025. [Online]. Available: https://arxiv.org/abs/2509.13516

[39] A. S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of AI deployment?" in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*. Rio de Janeiro, Brazil: Association for Computing Machinery, 2024, pp. 85–99.

[40] K. C. Aashish *et al.*, "Towards eco friendly cybersecurity: machine learning based anomaly detection with carbon and energy metrics," *arXiv preprint arXiv:2601.00893*, 2026, instruments Logistic Regression, Random Forest, SVM, Isolation Forest and XGBoost with CodeCarbon and defines an F1-per-kilowatt-hour eco-efficiency index.

[41] M. Jay, V. Ostapenco, L. Lefèvre, D. Trystram, A.-C. Orgerie, and B. Fichel, "An experimental comparison of software-based power meters: Focus on CPU and GPU," in *Proceedings of the 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2023, pp. 106–118.

[42] R. Feldt and A. Magazinius, "Validity threats in empirical software engineering research – an initial survey," in *Proceedings of the 22nd International Conference on Software Engineering & Knowledge Engineering (SEKE)*, 2010, pp. 374–379.

[43] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, and A. Chatzigeorgiou, "Identifying, categorizing and mitigating threats to validity in software engineering secondary studies," *Information and Software Technology*, vol. 106, pp. 201–230, 2019.

[44] D. I. K. Sjøberg and G. R. Bergersen, "Construct validity in software engineering," *IEEE Transactions on Software Engineering*, vol. 49, no. 3, pp. 1374–1396, 2023.

[45] R. Verdecchia, E. Engström, P. Lago, P. Runeson, and Q. Song, "Threats to validity in software engineering research: A critical reflection," *Information and Software Technology*, vol. 164, p. 107329, 2023.

[46] V. R. Basili, G. Caldiera, and H. D. Rombach, "The goal question metric approach," in *Encyclopedia of Software Engineering*, vol. 1. John Wiley & Sons, 1994, pp. 528–532.

[47] T. D. Cook and D. T. Campbell, *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Boston: Houghton Mifflin, 1979.

[48] D. T. Campbell and J. C. Stanley, *Experimental and Quasi-Experimental Designs for Research*. Boston: Houghton Mifflin, 1963.

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778, arXiv:1512.03385.

[50] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *3rd International Conference on Learning Representations (ICLR)*, 2015, arXiv:1409.1556.

[51] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[52] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[53] Y. Le and X. Yang, "Tiny ImageNet visual recognition challenge," Stanford University, Tech. Rep. CS231N course report, 2015. [Online]. Available: http://cs231n.stanford.edu/reports/2015/pdfs/yle_project.pdf

[54] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action: Experiences in using RAPL for power measurements," *ACM Transactions on Modeling and Performance Evaluation of Computing Systems*, vol. 3, no. 2, pp. 9:1–9:26, 2018.

Leonardo Pampaloni graduated with a Master's degree in Software Engineering from the University of Florence, Italy, achieving top marks. His research interests lie in software sustainability and energy efficiency in Artificial Intelligence, with particular emphasis on Green AI and computer vision. He has contributed to empirical studies on cross-language benchmarking of AI frameworks and optimization of DL workloads. More information is available at https://github.com/Pampaj7.

Marco Pagliocca received a Bachelor's degree in Computer Engineering with highest honors from the University of Florence, Italy, where he is currently pursuing a Master's degree in the same field. His research interests focus on environmental sustainability, human-centered engineering, and the use of technology as a means to improve everyday life. He believes that Artificial Intelligence can simplify mechanical tasks and encourage greater attention to meaningful aspects of human activity.

Enrico Vicario is a Professor of Computer Science and Engineering and the Head of the Software Technologies Laboratory of the University of Florence, Italy. His research interests include the area of software engineering, with a focus on quantitative evaluation of stochastic models, software architectures and methodologies, and on their connection through model-driven engineering practices.

Roberto Verdecchia is an Assistant Professor at the Software Technologies Laboratory (STLab) of the University of Florence, Italy. He holds a double Ph.D. in Computer Science, appointed by the Gran Sasso Science Institute, L'Aquila, Italy, and the Vrije Universiteit Amsterdam, the Netherlands. His research interests focus on the adoption of empirical methods to improve software development and system evolution, with particular interest in the fields of technical debt, software architecture, software sustainability, and software testing. More information is available at robertoverdecchia.github.io.